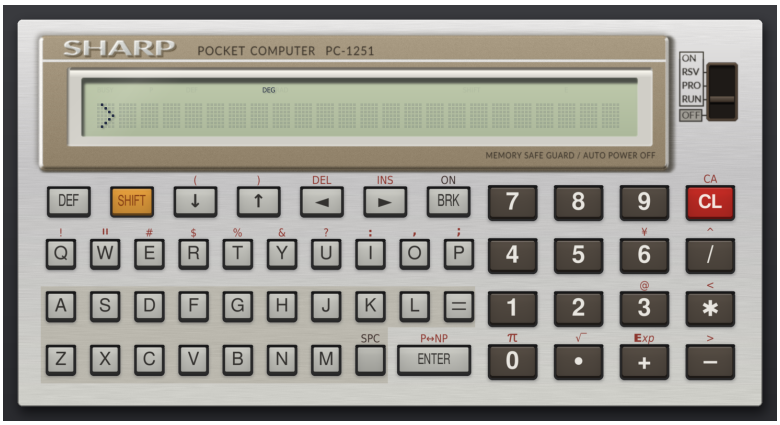
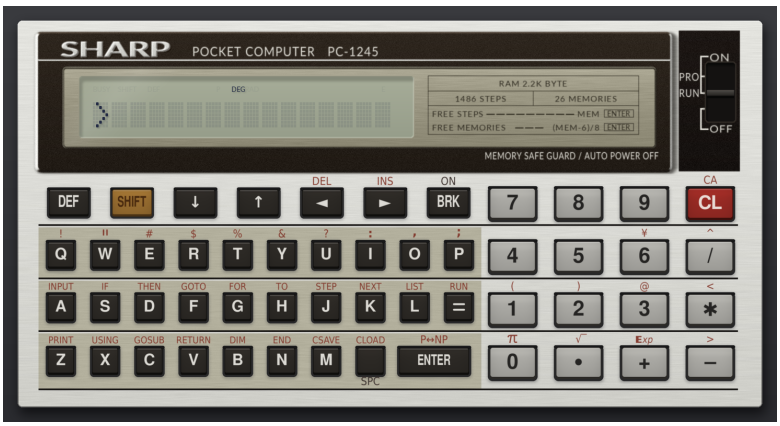
# SHARP PC-1251/PC-1245/PC-1261/PC-1350
# エミュレータ

使い方の手引き




おりぐち

2026 年 10 月 25 日 (エミュレータ 0.5.4 版)

このエミュレータは PC-1251、PC-1245、PC-1261、PC-1350 の ROM をそのまま動かします。ROM は付属していません (2.4 節)。ROM がなくても、マシン語の作業欄は使えます (6 節)。

# 目次


1 はじめに 2

2 インストールと起動 3
2.1 端末を開く . . . 3
2.2 uv をインストールする . . . 3
2.3 エミュレータのフォルダを用意する . . . 3
2.4 ROM を用意する . . . 4
2.5 起動する . . . 5
2.6 終わる . . . 6
2.7 うまく動かないとき . . . 6

3 画面と操作 7
3.1 各部の名前 . . . 7
3.2 マウスで操作する . . . 8
3.3 パソコンのキーボードで操作する . . . 8
3.4 エミュレータの操作 . . . 9
3.5 状態を保存する (どこでもセーブ) . . . 11
3.6 遊んでいるところを録画する . . . 11
3.7 モードスイッチ . . . 12
3.8 SHIFT と DEF . . . 12

4 使ってみる 12
4.1 電卓として使う . . . 12
4.2 BASIC のプログラムを打ち込んで動かす . . . 13
4.3 打ち込みを楽にする、行を直す・消す . . . 14
4.4 PRINT で止めずに見せる (WAIT) . . . 15
4.5 DEF キーで実行する . . . 16
4.6 RSV モードで文字列を登録する . . . 16
4.7 エラーが出たら . . . 17
4.8 電源とメモリ . . . 17

5 プログラムのファイルを使う 18
5.1 置き場所に置いて、一覧から読み込む . . . 18
5.2 ファイルの形式 . . . 20
5.3 端末から読み込む . . . 23
5.4 マシン語を動かす . . . 23
5.5 CALL で表示をつけたまま計算する . . . 23
5.6 マシン語で液晶に描く . . . 24
5.7 PC-インタープリタともぐらたたき . . . 26
5.8 ほかの読み込み方 . . . 27
5.9 プログラムをファイルに書き出す . . . 27
5.10 プリンタに印字する . . . 27
5.11 カセットの音 (wav) で実機とやり取りする . . . 28

6 マシン語の作業欄 29
6.1 ROM なしで起動する . . . 30
6.2 作業欄の各部 . . . 31
6.3 ダンプリストの読み方 . . . 32
6.4 ダンプを打ち込む . . . 32

6.5 実行して調べる . . . 34
6.6 命令とデータを見分ける . . . 35
6.7 打ち込んだマシン語をファイルに書き出す . . . 36
6.8 ROM のある本体で作業欄を使う . . . 37

7 PC-1250・PC-1255 として使う 37

8 PC-1245 として使う 38
8.1 液晶 . . . 38
8.2 モードスイッチ . . . 39
8.3 キー . . . 39
8.4 メモリと見本のプログラム . . . 39

9 PC-1261 として使う 40
9.1 液晶 . . . 40
9.2 キー . . . 41
9.3 メモリと見本のプログラム . . . 41
9.4 CPU の速さ . . . 42

10 PC-1350 として使う 42
10.1 電源とモード . . . 43
10.2 液晶 . . . 43
10.3 キー . . . 43
10.4 メモリと見本のプログラム . . . 44

付録 A 起動のオプション 45

付録 B SHIFT で入る記号 45

付録 C メモリマップと入出力 46

付録 D 実機と違うところ 47

付録 E ファイルの一覧 48

付録 F 謝辞 48

# 1　はじめに

このエミュレータは、シャープのポケットコンピュータ PC-1251(1982 年発売、29,800 円) と PC-1245(1983 年発売、17,800 円)、PC-1261、PC-1350 を、パソコンの上で動かすものです。同じ ROM で、RAM の大きさだけが違う PC-1250・PC-1255、PC-1260・PC-1262 としても動きます。CPU の SC61860 を命令単位で動かし、実機の内部 ROM と BASIC ROM をそのまま実行します。キーとモードスイッチはマウスでも操作できます。

BASIC のプログラムを打ち込んで動かせるほか、電卓としても使えます。BEEP の音も鳴ります。BASIC のテキストや 16 進ダンプのファイルを決まったフォルダに置いておけば、一覧から選んで読み込めますし、マシン語を RAM に書いて `CALL` で呼ぶこともできます。実機の「MEMORY SAFE GUARD」と同じく、終了してもプログラムは消えません。ROM のファイルがなくても、マシン語を打ち込んで実行し、1 命令ずつ調べられる作業欄があります (6 節)。

どの機種にするかは起動のときに選び、何も指定しなければ PC-1251 になります。この手引きは PC-1251 を例に書き、ほかの機種で違うところは機種ごとの節 (PC-1250・PC-1255 は 7 節、PC-1245 は 8 節、PC-1261 は 9 節、PC-1350 は 10 節) にまとめました。

## 動作環境

- macOS、Windows(10 以降)、Linux。
- Python 3.10 以降。Python を別にインストールする必要はありません。次の節でインストールする uv が、必要な版の Python とライブラリをまとめて用意します。
- 初回の起動でライブラリを入れるときだけ、インターネットにつながっている必要があります。

## キーとコマンドの書き方

パソコンのキーは Esc、本体のキーは SHIFT のように書き分けます。command / Ctrl + O は、macOS では command を、Windows と Linux では Ctrl を押しながら O を押すという意味です。macOS と Windows で違うところは「macOS」「Windows」と見出しをつけて両方を書きました。端末 (macOS の「ターミナル」、Windows の「PowerShell」) で打つコマンドは、次のような枠に入れてあります。行の頭の$や>は打たなくてかまいません。

```
$ uv run pc1251
```

# 2　インストールと起動

## 2.1　端末を開く

■macOS　「アプリケーション」→「ユーティリティ」→「ターミナル」を開きます。Spotlight(command+Space) で「ターミナル」と打っても開けます。

■Windows　スタートメニューで「PowerShell」と打ち、「Windows PowerShell」を開きます。「ターミナル」(Windows Terminal) が入っていれば、「ターミナル」を使ってもかまいません。

## 2.2　uv をインストールする

uv は Python とライブラリをまとめて扱う道具です。一度入れれば、次からは入れ直す必要はありません。

■macOS

```
$ curl -LsSf https://astral.sh/uv/install.sh | sh
```

■Windows

```
> powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

入れ終わったら、端末をいったん閉じて開き直します (新しく入ったコマンドの場所を端末に読み込ませるため)。次のコマンドでバージョン番号が出れば成功です。

```
$ uv --version
```

**Homebrew や winget を使っている場合**

macOS で `brew install uv`、Windows で `winget install --id=astral-sh.uv -e` としても同じ uv が入ります。

## 2.3　エミュレータのフォルダを用意する

GitHub のページから `git clone` で取ってくるか、「Code」→「Download ZIP」でダウンロードした ZIP ファイルを展開して、好きな場所に置いてください。`git clone` で取ってきたときのフォルダの名前は `pc1251-emulator` で、以下でも `pc1251-emulator` と書きます。端末で `pc1251-emulator` に移ってからコマンドを打ちます。

■macOS　(ホームの `src` の下に置いた場合)

```
$ cd ~/src/pc1251-emulator
```

■Windows　(「ドキュメント」の下に置いた場合)

```
> cd $HOME\Documents\pc1251-emulator
```

フォルダの場所を打つのが面倒なら、cd と空白を打ってから、Finder やエクスプローラーのフォルダをターミナルや PowerShell のウィンドウにドラッグ&ドロップすると、場所が入ります。

## 2.4　ROM を用意する

ROM はシャープの著作物なので、エミュレータには含めていません。実機から読み出したものを使ってください。

機種ごとに 2 つのファイルが要ります。エミュレータのフォルダの中に rom フォルダを作り、表のファイル名で置きます (置き方の例は表のあと)。使う機種のぶんだけあれば動き、両方を並べて置いてもかまいません。

| ファイル名 | 中身 | 大きさ |
|---|---|---|
| cpu-1251.rom | PC-1251 の CPU(SC61860) の内部 ROM、0000〜1FFF 番地 | 8192 バイト |
| bas-1251.rom | PC-1251 の BASIC ROM、4000〜7FFF 番地 | 16384 バイト |
| cpu-1245.rom | PC-1245 の CPU の内部 ROM、0000〜1FFF 番地 | 8192 バイト |
| bas-1245.rom | PC-1245 の BASIC ROM、4000〜7FFF 番地 | 16384 バイト |
| cpu-1260.rom | PC-1260/1261/1262 の CPU の内部 ROM、0000〜1FFF 番地 | 8192 バイト |
| bas-1260.rom | PC-1260/1261/1262 の BASIC ROM、8000〜FFFF 番地 | 32768 バイト |
| cpu-1350.rom | PC-1350 の CPU の内部 ROM、0000〜1FFF 番地 | 8192 バイト |
| bas-1350.rom | PC-1350 の BASIC ROM、8000〜FFFF 番地 | 32768 バイト |

たとえば macOS でホームの src の下に置いた場合は、次のようになります。

```
~/src/pc1251-emulator/
    pc1251emu/
    programs/
    rom/
        cpu-1251.rom
        bas-1251.rom
        cpu-1245.rom      (PC-1245 も使うとき)
        bas-1245.rom
        cpu-1260.rom      (PC-1261 も使うとき)
        bas-1260.rom
```

rom フォルダは、エミュレータのフォルダに移ってから次のコマンドで作れます (macOS でも

Windows でも同じです)。Finder やエクスプローラーで ROM のファイルを `rom` フォルダへ入れます。

```
$ mkdir rom
```

Windows で「ドキュメント」の下に置いた場合は、`C:\Users\名前\Documents\pc1251-emulator\rom\cpu-1251.rom` のようになります。

実機からの読み出し方は、大阪工業大学のサイトにある yagshi 氏のページ「Extracting ROM Data (ROM 吸い出し)」(https://www.oit.ac.jp/rd/labs/kobayashi-lab/~yagshi/old_web/misc/pocketcom/extrom.html) に、PC-1251 を例にした手順があります。BASIC ROM は `CSAVE M` で区切りながらカセットに保存し、内部 ROM は短いマシン語で外に写してから保存します。録音した音をパソコンでバイナリに戻し、つなげて 1 つのファイルにします。PC-1245 は、ht-deko.com のページ「SHARP PC-1245/50/51/55 の使い方」(https://ht-deko.com/pc1245/) に、BASIC のリストつきの手順があります。内部 ROM は短いマシン語で 1KB ずつ RAM に写してから、BASIC ROM は 4KB ずつ、どちらも `CSAVE M` で保存します。

ROM には版の違いがあるので、エミュレータが確かめるのはファイルの大きさだけです。ファイルが見つからないときや大きさが足りないときは、ROM なしで起動して、マシン語の作業欄を出します (6.1 節)。PC-1350 は、ROM なしでは起動できないので、メッセージを出して終わります。

**初めて uv run を使うとき**

最初の `uv run` では、uv が Python と、使うライブラリ (pygame、Pillow、Typer、録画に使う imageio-ffmpeg) を取ってきて入れるので、しばらく時間がかかります。2 回目からはすぐに始まります。ライブラリはエミュレータのフォルダの中の `.venv` に入ります (Python の本体と、取ってきたファイルの控えは、uv が自分のフォルダにしまいます)。

ROM を `rom` 以外のフォルダに置くときは、環境変数 `PC1251_ROM` にフォルダの場所を入れておきます。ほかに、ホームの `.pc1251` フォルダも探します。

## 2.5　起動する

```
$ uv run pc1251
```

PC-1251 の画面がウィンドウに出て、液晶に>(プロンプト) が出れば動いています。最初の 8 秒ほどは、画面の中ほどにキーの説明 (パソコンのキーとポケコンのキーで対応が違うもの) が出ます。何かキーを押すかクリックすると消えます (command / Ctrl + / でいつでも出し入れできます)。

PC-1245 として起動するときは、`--model 1245` を付けます (8 節)。PC-1261 として起動するときは`--model 1261` です (9 節)。

```
$ uv run pc1251 --model 1245
$ uv run pc1251 --model 1261
```

画面が大きすぎたり小さすぎたりするときは、拡大率を指定します。画面に収まらないときは自動で縮めます。

```
$ uv run pc1251 --scale 0.8
```

画面の文は英語にもできます。操作欄の「表示・入力」のメニューで「English」を選ぶと、すぐに英語になります (英語の画面で「日本語」を選ぶと戻ります)。選んだ言語は覚えておき、次に起動したときも使います。起動のときに`--lang en`か`--lang ja`を付けるか、環境変数`PC1251_LANG`に`en`か`ja`を書いておくと、メニューで選んだものより先にそちらを使います。

どれもなければ、パソコンの言語の設定 (環境変数のロケール、ないときは macOS のシステム設定の言語か Windows の表示の言語) が日本語なら日本語、ほかの言語なら英語で出し、わからなければ日本語にします。キーや液晶の字、BASIC の命令、見本のプログラムの名前は、どちらの言語でも同じです。

```
$ uv run pc1251 --lang en
```

## 2.6　終わる

ウィンドウを閉じれば終わります。終わるときには、実機のスイッチを OFF にしたのと同じ手順で電源を切ってから、RAM の中身を保存します (4.8 節)。次に起動すると、プログラムも変数もそのまま残っています。

保存したものを使わずにまっさらな状態で始めたいときは`--fresh`をつけます。`--fresh`をつけたときは、終わるときにも保存しません。

```
$ uv run pc1251 --fresh
```

## 2.7　うまく動かないとき

液晶に>が出ず、右に「マシン語 (ROM なし)」の欄が出る

ROM のファイルが見つからないか、大きさが足りないので、ROM なしで起動しています (6.1 節)。2.4 節を見て、`rom`フォルダに置いてください。PC-1350 では、ROM がないと`cpu-1350.rom`が見つかりませんと出て終わります。

uv というコマンドがないと言われる

uv を入れたあと、端末を開き直していません。閉じて開き直してください。

Windows で「スクリプトの実行が無効」と出る

uv をインストールするコマンドを、上に書いたとおり`-ExecutionPolicy ByPass`をつけて打ってください。

キーを打っても字が入らない

日本語入力になっていると、打ったキーがエミュレータに伝わりません。英数入力に切り替えてください。

Mac で F1〜F12 を押しても反応しない

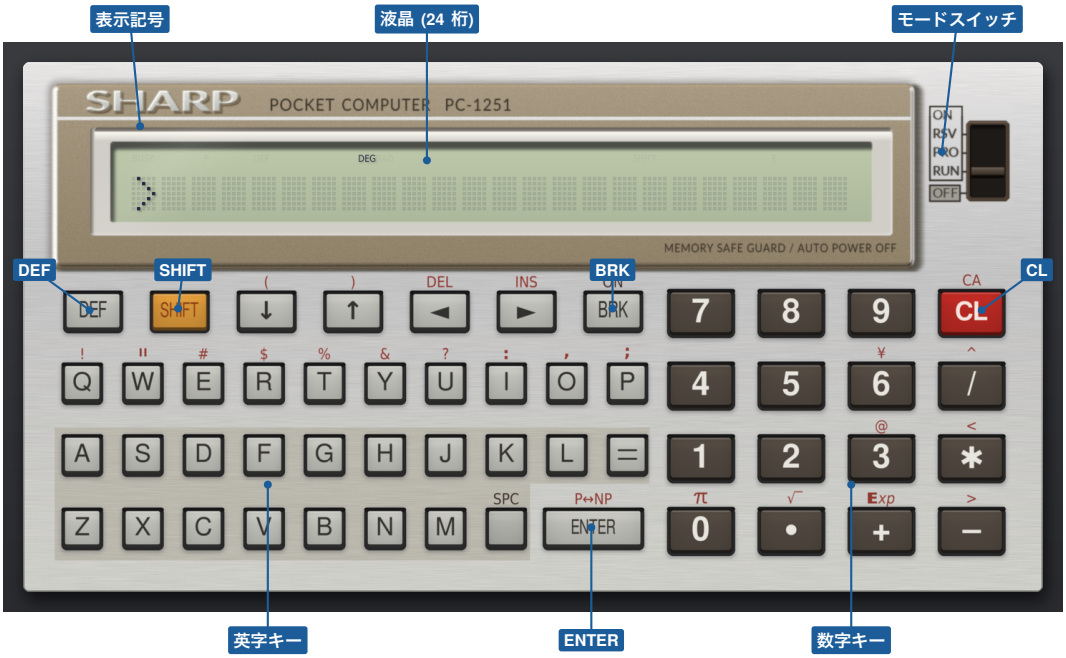
ノート型の Mac では、F1〜F12 は明るさや音量のキーになっていることがあります。fn を押しながら押すか、3.4 節の command / Ctrl + O などの組み合わせか、本体の下の操作欄のボタンを使ってください。

音がいらない

音を止めたいときは`--mute`をつけて起動します。

# 3　画面と操作

## 3.1　各部の名前




液晶

5×7 ドットの文字が 24 桁並びます。点いたドットは濃い青紫で、消えたドットもうっすら見えます。反射型の液晶なので、文字の影が奥に少しずれて落ちて見えます。

表示記号

液晶の上の小さな字です。DEG(角度の単位)、SHIFT(SHIFT を押した)、DEF、BUSY(計算中・実行中)、E(エラー) などが出ます。いまのモードは液晶には出ないので、スイッチの位置で見ます。

モードスイッチ

右上の縦のスイッチです。上から RSV、PRO、RUN、OFF。いちばん上の ON はスイッチの位置で

はなく、BRKキーが ON(電源を入れる) を兼ねるという表示です。

## 3.2　マウスで操作する

- 画面のキーをクリックすると、本体の同じキーを押したことになります。ボタンを押しているあいだはキーも押しっぱなしです。
- モードスイッチは、ラベル (RSV〜OFF) かつまみのあたりをクリックすると、クリックした位置に動きます。つまみを上下にドラッグしても動かせます。

## 3.3　パソコンのキーボードで操作する

英字・数字・Enter・Space・矢印キーは、PC-1251 の同じキーとして働きます。押しているあいだは押しっぱなしになります。記号は、パソコンで打った字を PC-1251 での打ち方 (SHIFTのあとにキー) に直して打ち込みます。たとえば:を打つと、SHIFT Iと押したことになります。

PC-1251 の ROM は、1 つのキーを受け取るのに 0.1 秒ほどかかります (キーを離してから次のキーまでに、何も押されていない時間が要る)。そこでエミュレータは、ROM が入力を待っているあいだは、パソコンで打ったキーを順番に並べて 1 つずつ届けます。ROM が入力を待っているところへキーを届けるあいだだけは本体を 8 倍の速さで動かすので、速く打ったときの取りこぼしと表示の遅れを減らしています。押し続けたときの繰り返し (押してから 0.7 秒ほどで始まる) は実機と同じ速さのままです。待たせすぎたキー (0.6 秒以上) は捨てるので、あとから遅れて流れてくることはありません。

プログラムが動いているあいだ (INKEY$やマシン語でキーを読むゲームなど) は、キーを並べずに、パソコンのキーを押したときにすぐ本体のキーを押し、離したときに離します。いくつかのキーを同時に押すこともできます (DEFを押したまま6を押すなど)。とても短く押したキーも、プログラムが読み落とさないように 0.05 秒 (PC-1261・PC-1350 では 0.08 秒) は押しておきます。

| macOS | Windows | PC-1251 | 説明 |
|---|---|---|---|
| 英数字・Enter・矢印 | 同じ | 同じキー | 押しているあいだは押しっぱなし |
| Space | Space | SPC | |
| 記号 | 記号 | SHIFT+ キー | 打った字に合わせて直す (付録 B) |
| Tab | Tab | SHIFT | |
| Option | Alt | DEF | |
| Delete (fn + delete) | Delete | CL | 表示を消す |
| delete | Backspace | ◀ と DEL | 1 字戻って消す |
| Esc | Esc | BRK | 実行を止める。電源が切れているときは ON |
| \ | \ | SHIFT 6 | ¥を打つ (ポケコンにはバックスラッシュがない) |

## 3.4　エミュレータの操作

モードスイッチの切り替えや一覧の表示など、PC-1251 のキーにないエミュレータの操作は、command (Windows と Linux では Ctrl) との組み合わせで行います。

| macOS | Windows | すること |
|---|---|---|
| command + ↑ ↓ | Ctrl + ↑ ↓ | モードスイッチを 1 段上・下へ動かす |
| command + O | Ctrl + O | プログラムの一覧 (5 節) |
| command + R | Ctrl + R | 裏の RESET ボタンを押す |
| command + T | Ctrl + T | 速くする (1/2 倍、1 倍、2 倍、4 倍、8 倍) |
| shift + command + T | Shift + Ctrl + T | 遅くする |
| command + E | Ctrl + E | BASIC のプログラムをファイルに書き出す (5.9 節)。作業欄にキーを送っているときは、作業欄のダンプを書き出す (6.7 節) |
| command + S | Ctrl + S | 画面を画像で保存する |
| shift + command + S | Shift + Ctrl + S | 状態を保存する (3.5 節) |
| shift + command + L | Shift + Ctrl + L | 保存した状態に戻す |
| shift + command + R | Shift + Ctrl + R | 録画を始める・止めて保存する (3.6 節) |
| shift + command + P | Shift + Ctrl + P | 録画を一時停止する・続ける |
| command + P | Ctrl + P | 実行を一時停止する・再開する |
| command + / | Ctrl + / | キーの説明を出す・消す |
| command + V | Ctrl + V | クリップボードの文字を打ち込む。作業欄にキーを送っているときは、ダンプを作業欄に貼り付ける (6.4 節) |
| command + K | Ctrl + K | 矢印キーを数字キーの 8・2・4・6 にする・戻す |
| command + D | Ctrl + D | マシン語の作業欄を出す・しまう (6.8 節) |
| command + Z | Ctrl + Z | 作業欄の書き込みを取り消す (shift も押すとやり直す。6.4 節) |
| command + A | Ctrl + A | 作業欄のカーソルの番地を開始番地にする (6.5 節) |

同じことは、本体の絵の下の操作欄からもできます。操作欄の左には、目的ごとのメニューを出すボタンが並びます。押すと、そのボタンのすぐ上にメニューが出ます。項目はクリックするか、↑ ↓ で選んで Enter で実行します。メニューを開いたまま操作欄のほかのボタンを押すと、メニューを閉じてそのボタンの操作をします (同じボタンなら閉じるだけ)。← → で隣のボタンのメニューへ移れます。

| ボタン | メニューの中身 |
|---|---|
| プログラム | プログラムの一覧、ファイルに書き出す、クリップボードから打ち込む、置き場所のフォルダを開く、作業欄の書き込みの取り消しとやり直し (作業欄を出しているとき)、読み込んだプログラムの説明 |
| 本体 | いまの状態を保存、保存した状態に戻す、プログラムを読み込む前に戻す、スイッチ、RESET ボタン |
| 表示・入力 | キーの説明、矢印キーの割り当て、マシン語の作業欄を出す・しまう (ROM があるとき)、画面を画像で保存、画面の言語 (English・日本語)、印字の保存先を開く、プリンタの紙をしまう |

続く「実行を一時停止」は CPU の実行を止めるボタン、「速さ」は実行の速さを選ぶボタン (いまの速さが出ます) です。右端の「● 録画」のボタンで、録画を始める・一時停止する・止めることができます (3.6 節)。ボタンにマウスを載せると、何をするボタンかとショートカットが、ボタンの上に出ます。ウィンドウが狭いときは短い表示になり、それでも入らなければ「速さ」のボタンを省きます (command / Ctrl + T と shift + command / Ctrl + T では変えられます)。操作欄の右には、機種と、最後に読み込んだプログラムの名前、いつもと違う設定 (矢印キーを数字キーにしているか、テープをセットしているか) が出ます。

ウィンドウの上で右クリックすると (Mac では control を押しながらクリックしても)、実行の一時停止、プログラムの一覧、画面を画像で保存、キーの説明だけの短いメニューが出ます。プリンタの紙の上で右クリックすると、紙の操作 (印字の保存先を開く、紙をしまう) が出ます。




command / Ctrl + P か操作欄の「実行を一時停止」を押すと、本体の時計を止めます。CPU も、ファイルの打ち込みも、その場で止まり、音も止まります。一覧や説明を読むあいだにゲームを進めたくないときに使います。止めているあいだは、同じ場所のボタンが「実行を再開」に変わり、ウィンドウの左上に「実行を一時停止中」と出ます。「実行を再開」か、もう一度 command / Ctrl + P を押すと続きから動きます。録画の一時停止 (shift + command / Ctrl + P) とは別の操作で、録画しているときに実行を止めると、止めた画面がそのまま動画に入ります。

**テンキーのないパソコンでゲームをするとき**

PC-1251 のゲームには、数字キーの 8 2 4 6 (5 を真ん中にした上下左右) で動かすものがあります。テンキーのないパソコンでは、command / Ctrl + K (または F7) を押すと、↑ ↓ ← → がそれぞれ 8 2 4 6 を押すようになります。もう一度押すと元に戻ります。プログラムを打ち込むときは、↑ ↓ が PC-1251 の ↑ ↓ として使えなくなるので戻しておいてください。起動のときに--numpad を付けると、矢印キーが数字キーになった状態で始まります。

**速さを変えたときの音**

1 倍以外の速さでは、音の高さも速さの倍率だけ変わります。2 倍なら BEEP は 1 オクターブ高く聞こえます。8 倍では音が高くなりすぎて、本来と違う高さに聞こえることがあります。

**ファンクションキー**

F1〜F4(スイッチの RUN、PRO、RSV、OFF)、F5(RESET)、F6(一覧)、F7(矢印キーを数字キーに)、F9(速くする。Shift+F9で遅くする)、F10(説明)、F12(画面の保存) も使えます。ただ、Mac では fn を押しながらでないと明るさや音量のキーになり、Windows でもほかのアプリに取られることがあるので、ふだんは上の組み合わせか操作欄のボタンを使ってください。

command/Ctrl+Sで保存した画像は、ホームの`.pc1251`フォルダに`shot001.png`、`shot002.png`……の名前で置かれます (Windows では`C:\Users\名前\.pc1251`)。

## 3.5　状態を保存する (どこでもセーブ)

shift+command/Ctrl+Sを押すと、そのときの RAM、CPU のレジスタ、ポート、タイマ、スイッチの位置をまとめて保存します。shift+command/Ctrl+Lを押すと、保存したところに戻って、そこから続けられます。ゲームの途中で一休みするときや、録画を撮り直したい場面の手前で使います。操作欄の「本体」のメニューの「いまの状態を保存」「保存した状態に戻す」でも同じです。「保存した状態に戻す」の下には、保存した日時が出ます。

保存先はホームの`.pc1251`フォルダの`state-1251.json`(PC-1245 なら`state-1245.json`のように機種ごと) です。保存し直すと上書きし、1 つ前の状態は`state-1251.json.bak`に残します。ほかの機種の状態や、保存したときと違う ROM の状態、壊れたファイルは読み込まず、いまの状態もそのままにします。最後に読み込んだプログラムの書き出し用の情報 (題名、`# run:`、`# help:`、ダンプの指示など) も一緒に保存し、状態を戻すとその情報も保存したときのものに戻ります。元のファイルをあとで書き換えても、書き出すファイルに書き換えたあとの情報が付くことはありません。

テープ (5.11 節) の再生の位置と、ファイルの読み込みで打ち込んでいる途中の文字は保存しません。状態を戻すと、セットしていたテープは外れます。起動のときに`--state`ファイルをつけると、保存した状態から始めます。

## 3.6　遊んでいるところを録画する

操作欄の右端の「● 録画」を押すか、shift+command/Ctrl+Rを押すと録画を始めます。録画しているあいだは、そのボタンが「録画中」と経過時間の表示に変わります。このボタンを押すと、すぐ上に停止と一時停止のメニューが出ます。「録画を止めて保存」(またはshift+command/Ctrl+R) で録画を止めて、動画のファイルに保存します。「録画を一時停止する」(またはshift+command/Ctrl+P) を選ぶと、一時停止しているあいだの画面と音は動画に入りません。「録画を再開する」で続きを録ります。録画しているあいだは、ウィンドウの右上に「録画中」(一時停止しているときは「録画を

一時停止中」) と経過時間が出ます。この表示と、画面の下に出る知らせやメニューは、録画には入りません。

動画は 1280×720 の MP4 のファイルで、上に液晶を拡大した絵、その下に本体全体の絵が並びます。押しているキーは、本体の絵の中で押された形になります。左上には、最後に読み込んだプログラムの名前が入ります。音も入ります。録画の音は、パソコンのスピーカーで鳴らしている音とは別に、CPU がブザーへ出した信号から作るので、パソコンの音を消していても、`--mute` で起動したときでも入ります。速さを 1 倍以外にしているときは、鳴らす音と同じく、録画の音の高さも速さの倍率だけ変わります。

ファイルはプログラムの置き場所 (`~/.pc1251/programs`) に「rec-日時.mp4」の名前で置かれます。録画中にエミュレータを終了したときは、終了するまでの分を保存します。プログラムを読み込んでいるあいだ (全速で打ち込んでいるとき) は、画面だけを録り、音は入れません。

動画を作るのには、ffmpeg というプログラムを使います。ffmpeg は、エミュレータを動かすときに uv が imageio-ffmpeg のライブラリと一緒にインストールするので、別にインストールする必要はありません (パソコンに ffmpeg が入っていれば、そちらを使います)。ただし、imageio-ffmpeg が ffmpeg を含むのは、Windows、macOS、Linux(x86_64 と aarch64) だけです。ほかの環境では、ffmpeg を別にインストールして PATH に置かないと録画できません。ffmpeg が見つからないときは録画を始めず、画面の下に「ffmpeg が見つかりません」という警告を出します。

### 3.7　モードスイッチ

| モード | すること |
|---|---|
| `RUN` | 計算とプログラムの実行。 |
| `PRO` | プログラムを打ち込む・直す・`LIST` で見る。 |
| `RSV` | よく使う文字列をキーに登録する (4.6 節)。 |
| `OFF` | 電源を切る。プログラムと変数は残る。 |

### 3.8　SHIFT と DEF

SHIFT は、押して離してから次のキーを押します (同時に押す必要はありません)。押すと表示記号 `SHIFT` が点き、次のキーがキーの上に赤く刷ってある字になります。たとえば SHIFT W で"、SHIFT 0 で $\pi$ です。

DEF は、プログラムの中の`"A"`のような印の行から実行を始めるキーです (4.5 節)。

## 4　使ってみる

### 4.1　電卓として使う

RUN モードで式 (たとえば `12*3`) を打って ENTER を押すと、答えが出ます。

```
12*3
```



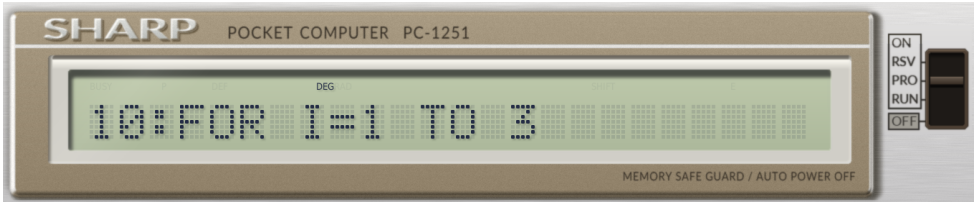

CL (パソコンでは Delete) で表示が消えます。

## 4.2　BASIC のプログラムを打ち込んで動かす

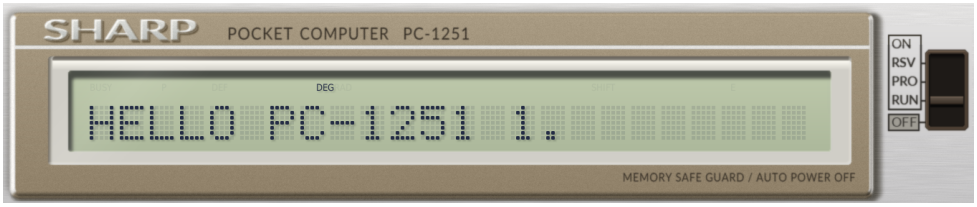
1. スイッチの PRO をクリックするか、command/Ctrl+↑で PRO モードにします。
2. 前のプログラムを消したければ NEW と打って ENTER を押します。
3. 行番号をつけて、1 行ずつ打ち込みます。行の終わりで ENTER を押します。

```
10 FOR I=1 TO 3
20 PRINT "HELLO PC-1251 ";I
30 NEXT I
40 END
```

4. ↓を押すと先頭の行が出て、押すたびに次の行へ進みます。↑で前の行に戻ります (何も出ていないときに押すと、最後の行が出ます)。LIST 30 のように打つと、30 行から見られます。




5. command/Ctrl+↓で RUN モードに戻り、RUN と打って ENTER を押します。




PC-1251 の PRINT は、表示したところで止まって待ちます。ENTER を押すと先へ進みます (止めずに進める方法は次の節)。実行を途中で止めるには BRK (パソコンでは Esc) を押します。

長いプログラムを手で打つのは大変なので、ふつうはテキストファイルに書いておき、5 節の方法で読み込むのが楽です。

**ENTER のあとは少し待つ**

実機と同じく、ENTER を押したあとは ROM が行を中間コードに直すのに時間がかかります。パソコンのキーで打つときは、エミュレータが ROM の入力待ちに戻るのを待ってから次のキーを届けるので、取りこぼしません。ファイルから読み込むときは、エミュレータが行ごとに直し終わるのを待ってから次の行を打ち込みます。

**IF のあとの代入には LET がいる**

IF の条件のあとに代入を書くときは、LET を省けません。THEN をつけても同じです。

| 書き方 | 結果 |
|---|---|
| `IF S>H LET H=S` | 代入される |
| `IF S>H THEN LET H=S` | 代入される |
| `IF S>H THEN H=S` | ERROR 4 |
| `IF S>H H=S` | エラーは出ないが、代入されない |

最後の書き方は止まらずに進んでしまうので、間違いに気づきにくいところです。代入でなければ、`IF A=1 GOTO 50` や `IF D<>1 POKE &C5DF,0` のように、条件のすぐあとに命令を書けます。

## 4.3　打ち込みを楽にする、行を直す・消す

■命令は略して打てる　命令の頭の何文字かのあとに.(ピリオド) を打つと、打った文字で始まる命令になります。たとえば `P."AB"` と打てば `PRINT "AB"` と同じです。PRO モードでも RUN モードでも使え、LIST すると元の綴りで表示されます。

| 略記 | 命令 | 略記 | 命令 | 略記 | 命令 |
|---|---|---|---|---|---|
| P. | PRINT | G. | GOTO | GOS. | GOSUB |
| I. | INPUT | F. | FOR | N. | NEXT |
| RE. | RETURN | REA. | READ | RES. | RESTORE |
| R. | RUN | L. | LIST | T. | THEN |
| B. | BEEP | W. | WAIT | CA. | CALL |
| D. | DIM | E. | END | S. | STOP |

同じ文字で始まる命令がいくつかあるときは、ROM の命令の表で先に載っているものになります。C. は CALL ではなく CONT に、RE. は READ ではなく RETURN になるので、ほしい命令にならないときは文字を 1 つ増やします。.bas のファイルに略記で書いても同じように読み込め、5.9 節の方法で書き出すと元の綴りに戻ります。

■行を消す　PRO モードで行番号だけを打って ENTER を押すと、打った番号の行が消えます。たとえば `20` ENTER で 20 行が消えます。プログラム全体を消すときは NEW です。同じ行番号で打ち直すと、前の行と入れ替わります。

■行を直す　PRO モードで `LIST 30` のように行を出し、▶ を押すと、行番号のすぐあとの文字にカーソルが出ます。

- ◀▶でカーソルを動かし、文字を打つとカーソルの位置の文字を上書きします。
- SHIFT ▶(INS) でカーソルの位置に 1 字ぶんの空きを作り、空いたところへ文字を打ち込みます。
- SHIFT ◀(DEL) でカーソルの位置の文字を消します。
- 直し終わったらENTERを押すと、直した行に入れ替わります。
- ↓↑で次の行・前の行を出せます。

■パソコンのキーボードで打つとき　Backspace(macOS ではdelete) を押すと、エミュレータが◀と SHIFT ◀(DEL) を続けて押し、カーソルの 1 つ前の文字を消します。打ち間違えたらそのままBackspaceで戻れます。矢印キーは◀▶↑↓、Delete(macOS ではfn+delete) はCL(表示を消す) です。

## 4.4　PRINT で止めずに見せる (WAIT)

PRINT のたびにENTERを押すのでは、表示を動かすことができません。WAIT を使うと、PRINT は決まった時間だけ表示してから、止まらずに次へ進みます。

| 書き方 | PRINT の動き |
|---|---|
| WAIT 64 | 表示してからおよそ 1 秒待ち、次へ進む。数は 10 進法で、大きいほど長く見せる |
| WAIT 0 | 待たずにすぐ次へ進む。表示は目に見えないほど短い |
| WAIT | 数を書かないと、ENTERを待つもとの動きに戻る |

一度 WAIT を実行すると、以降の PRINT はすべて同じ待ち時間で動きます。次の例は、POCKET COMPUTER の文字を右から左へ 1 字ずつ流し、左端まで来たところで止まります (見本の anime.bas)。

```
10 DIM T$(0)*40
20 T$(0)="                          POCKET COMPUTER"
30 WAIT 2
40 FOR I=1 TO 24
50 PRINT MID$ (T$(0),I,24)
60 NEXT I
70 WAIT:PRINT "POCKET COMPUTER"
80 END
```




ふつうの文字列変数 (A$など) には 7 字までしか入らないので、10 行の DIM T$(0)*40 で、40

字まで入る文字列の配列 T$を作っています。かっこの中の 0 は添字の最大値で、使えるのは T$(0) の 1 つだけです (T$(1) を使うと ERROR 3 になります。DIM T$(1)*40 なら T$(0) と T$(1) の 2 つが使えます)。*40 は 1 つに入る字数で、書かなければ 16 字です。20 行では、T$(0) に空白を 24 字入れ、続けて流したい文字を入れておきます。50 行の MID$で I 字目から 24 字を切り出して表示し、I を 1 つずつ増やすと、文字が 1 字ずつ左へ動きます。30 行の WAIT 2 で、1 コマが 0.2 秒ほどになります (待ち時間のほかに、文字列の計算と表示の時間がかかる)。PC-1245 では左の 16 桁だけが見えるので、文字は見えない 17 桁目より右から入ってきます。

PC-1251 の BASIC には、液晶の点を 1 つずつ描く命令がありません。動きのある表示は、文字を PRINT し直して作ります。点で絵を描きたいときは、液晶の RAM に直接書くマシン語を使います (5.6 節の見本)。液晶に 1 列ずつ点を打てる PC-インタープリタ (5.7 節) の DEG も使えます。PC-インタープリタの PRINT は止まらないので、WAIT は時間待ちの命令として使います。

## 4.5　DEF キーで実行する

行の頭に"A"のような印をつけておくと、RUN モードで DEF A と押すだけで、印をつけた行から実行できます。

```
10 "A" PRINT 5
20 END
```

DEF A で 5 が出ます。印に使えるのは、RSV モード (4.6 節) と同じ灰色の帯の上の 18 個のキー (A〜L、=、Z〜M、SPC) です。

## 4.6　RSV モードで文字列を登録する

よく打つ BASIC の命令や式を、キーに登録しておけます。

1. スイッチの RSV をクリックして RSV モードにします。
2. SHIFT A と押すと、液晶に A: と出ます。
3. 登録したい文字列 (たとえば PRINT や 12*3) を打って、ENTER を押します。




4. RUN モードか PRO モードで SHIFT A と押すと、登録した文字列が打ち込まれます。

登録に使えるのは、キーボードの下 2 段の灰色の帯の上にある、A〜L と =、Z〜M と SPC の 18 個です。帯の外のキー (Q など) には登録できません。

## 4.7　エラーが出たら

打ち込んだ式やプログラムにまちがいがあると、ERROR と番号が出て、表示記号 E が点きます。CL で消えます。




**RUN モードで PRINT 3 と打つと ERROR 1 になる**

RUN モードで PRINT を式の前につけて直接打つと、ERROR 1 になります。エミュレータのまちがいではなく、ROM の動きです (ROM の 56BA 番地で状態を見て止めています)。電卓として使うときは 12*3 のように式だけを打ちます。プログラムの中の PRINT は、ふつうに動きます。

## 4.8　電源とメモリ

電源を切る

スイッチの OFF をクリックします。スイッチが OFF になったことを ROM が読んで、メモリが生きている印を書いてから自分で電源を切ります。

電源を入れる

スイッチを RUN か PRO に戻します。

オートパワーオフ

何もしないで 11 分ほどたつと、電源が切れます (実機と同じ)。BRK (ON) を押せば入ります。プログラムは残っています。

プログラムが残るしくみ

PC-1251 は、電源を入れたときに CPU の中のメモリに書いた印を確かめ、印がなければメモリがこわれたとみなして中身を消します。エミュレータは終わるときにも ROM に電源を切らせ、RAM(4KB) と CPU の中のメモリ (96 バイト) の両方を、ホームの .pc1251/ram.bin に保存します。

RESET ボタン

command / Ctrl + R で本体の裏の RESET ボタンを押せます。プログラムが暴走して BRK でも止まらないときに使います。

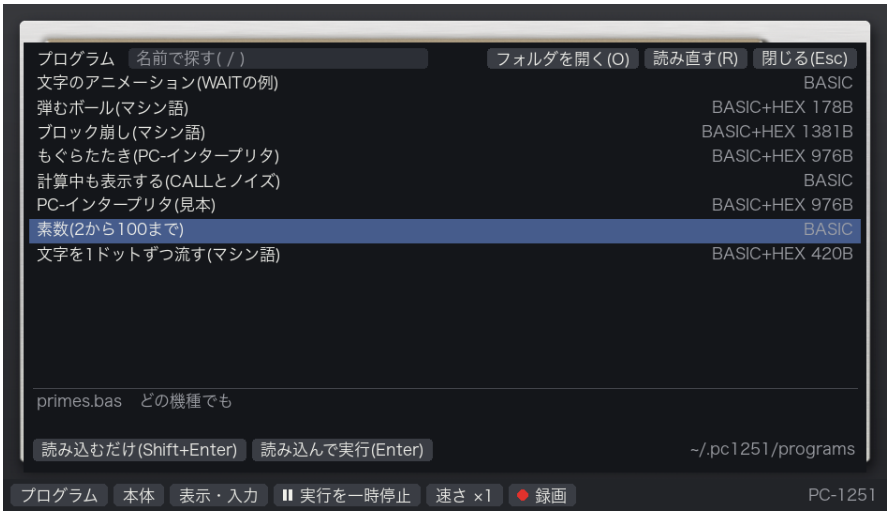
# 5　プログラムのファイルを使う

## 5.1　置き場所に置いて、一覧から読み込む

BASIC のプログラムやマシン語のダンプを、決まったフォルダ (置き場所) にファイルとして置いておくと、command / Ctrl + O で開く一覧に出てきます。一覧の行をクリックすると選ぶだけで、選んである行をもう一度クリックしても読み込みません。読み込むのは、下の「読み込んで実行」か「読み込むだけ」のボタン (または Enter、Shift + Enter) です。選んだ行の下には、ファイル名と動く機種、ファイルに書いた説明 (`# help:`) が出ます。一覧の上の欄をクリックして名前の一部を打つと、合うものだけに絞れます。右上の「フォルダを開く」「読み直す」「閉じる」は、一覧そのものの操作です。

| | |
|---|---|
| macOS | `/Users/名前/.pc1251/programs` |
| Windows | `C:\Users\名前\.pc1251\programs` |

置き場所のフォルダは、最初に一覧を開いたときにできます。一覧で O を押すと、Finder やエクスプローラーで置き場所のフォルダが開くので、ファイルを入れてください。名前が「.」で始まるフォルダは、ふだんは隠れていて見えにくいので、O で開くのが確実です。



| キー | すること |
|---|---|
| ↑ ↓ | 選ぶ (マウスでクリックしても選べる) |
| ホイール、PageUp PageDown | 一覧を動かす (入りきらないときは、右にバーと「▼」が出る) |
| Enter | 読み込んで実行する (探しているときも同じ) |
| Shift + Enter | 読み込むだけ (打ち込んだプログラムを直したいときなど) |
| O | 置き場所のフォルダを開く |
| R | 一覧を読み直す (ファイルを足したあと) |
| / | 名前で探す (続けて名前の一部を打つ。Esc で探すのをやめる) |
| Esc | 一覧を閉じる |

置き場所のほかに、エミュレータのフォルダの programs にある見本 (素数、文字のアニメーション、計算中の表示、文字を 1 ドットずつ流す、弾むボール、ブロック崩し、PC-インタープリタ、もぐらたたき、PC-1261 の 2 行の表示) も一覧に並びます。同じ名前のファイルが両方にあるときは、自分の置き場所のほうが使われます。

■機種ごとのフォルダ　プログラムは、動く機種に合わせて次のように置きます。

- 1 つの機種だけで動くものは、置き場所の中の機種のフォルダ (pc1251、pc1245、pc1261、pc1350) に置きます。pc1251 に置いたものは PC-1250 と PC-1255 でも、pc1261 に置いたものは PC-1260 と PC-1262 でも、マシン語の番地が RAM に収まり、BASIC の置き場所にかからなければ使えます。
- いくつかの機種で動くものは、置き場所の直下に置き、ファイルに# model: 1251,1245 のように動く機種を書きます。
- どの機種でも動く BASIC は、直下に# model: を書かずに置きます。

直下に置いたもので# model: のないものは、書き込むマシン語の番地と、BASIC の PEEK の番地 (数で書いたもの) が RAM か液晶に収まる機種で使えるとみます。たとえば C100 番地から置くマシン語は PC-1251 と PC-1245 用になり、PC-1261 の一覧には出ません。番地がどちらもなければ、どの機種でも使えます。

```
programs/
    primes.bas          どの機種でも
    pcint.bas           # model: 1251,1245
    mogura.bas          # model: 1251,1245
    pc1251/
        break.bas       PC-1251 用
        break.hex
    pc1245/
        break.bas       PC-1245 用
        break.hex
    pc1261/
        hello.bas       PC-1261 用 (PC-1260、PC-1262 でも)
    pc1350/
```

```
        hello.bas       PC-1350 用
```

一覧には、いま動かしている機種で使えるものだけが並びます。ほかの機種用のものは出さず、パスを指定して開いても「(PC-1251 用)」のように知らせて読み込みません。同じ `title` のものがいまの機種のフォルダにあれば、いまの機種のフォルダのものだけを出します。機種ごとに作り分けたプログラムは、同じ名前でそれぞれのフォルダに置いておけば、どの機種でも一覧の同じ名前から読み込めます。

■ドラッグ&ドロップ　Finder やエクスプローラーからファイルをエミュレータのウィンドウにドロップしても、同じように読み込めます。

## 5.2　ファイルの形式

| ファイル | 読み込み方 |
|---|---|
| 名前.bas | BASIC のテキスト。PRO モードにして NEW してから 1 行ずつ打ち込み、もとのモードに戻す。 |
| 名前.hex | 16 進ダンプ。番地 データの行を読んで、RAM にそのまま書き込む。 |
| 名前.bas と名前.hex | 同じ名前の 2 つは 1 本のプログラムとして扱い、ダンプを書いてから BASIC を打ち込む。マシン語と、マシン語を呼ぶ BASIC を組にしておける。 |

■BASIC のファイル　ふつうのテキストファイルです。文字コードは UTF-8(ASCII の範囲なら何でも同じ) で、改行は macOS・Windows どちらの形でもかまいません。小文字で書いても大文字に直して読み込みます。PC-1251 のキーにない字は捨てます。指数を表す E(SHIFT + で入る Exp の字) は英字の E とは別の字なので、ファイルの中では [E] か€ と書きます。たとえば $1 \times 10^{-8}$ は 1[E]-8 と書きます。1E-8 や 1e-8 と書くと英字の E になり、PC-1251 では「1×E − 8」(変数 E との掛け算) になります。円周率の $\pi$ (SHIFT 0 で入る字) は、ファイルの中でも $\pi$ と書きます。空の行と、#で始まる行は読み込みません。実機で 1 行に打てるのは行番号を含めて 79 字までですが、79 字より長い行も、途中まで打って ENTER を押し、行を呼び出して行末に続きを打ち足せば入ります。行を呼び出して直すときの入力欄も行番号と中身を合わせて 79 字までで、すでに入っている命令は 1 字、打ったばかりの命令の綴りは綴りの字数で数えます。行末に打ち足すとあふれる行も、命令の綴りだけを先に打ってから、残りの字 (変数、数、記号、文字列) を SHIFT ▶ (INS) で途中に挿入すれば入ります。ファイルの行は、こうした手順で実機に欠けずに入るものなら、79 字を超えていてもそのまま読み込みます。どうしても欠けてしまう行があるファイルは、読み込みません。

```
# title: 素数 (2 から 100 まで)
# PRINT で止まるので、ENTER で次を出す
10 PRINT 2
20 FOR N=3 TO 100 STEP 2
30 D=3
40 IF D*D>N GOTO 70
50 IF N=INT (N/D)*D GOTO 80
```

```
60 D=D+2:GOTO 40
70 PRINT N
80 NEXT N
90 END
```

■16 進ダンプのファイル　1 行に番地とデータを 16 進法で書きます。番地は 4 桁です。データは空白で区切っても続けて書いてもかまいません。雑誌のダンプリストのように行の終わりに: とチェックサムがついていれば、: から後ろは読み飛ばします。#か; で始まる行はコメントです。行の途中でも ; から後ろはコメントなので、ニーモニックを書き添えておけます。

```
# title: マシン語の見本
C300 02 41 37
C303 0102030405060708   ; 表の中身
```

アセンブラ YASM61860 が出力する 16 進ダンプ (c300 : 02 41 37 …の形の行) も、手を加えずに.hex として読めます。YASM61860 のダンプでは、各行の番地が 8 の倍数に切り下げてあり、データが行の途中の番地から始まるときは、手前の番地の分が空白で埋めてあります (1 バイトにつき空白 3 字)。エミュレータは、空白の字数から本当の書き始めの番地を求めて読みます。

■マシン語のバイナリ (.bin)　.bin のファイルには、マシン語のバイトが並んでいるだけで、どの番地に置くかは書かれていません。そこで、.bas のファイルに# bin: ファイル名 番地の行を書き、置く番地を指定して読み込みます。番地は&C300、0xC300、C300 のどの形式で書いてもかまいません (どれも 16 進表記です)。

```
# title: マシン語の見本 (バイナリ)
# bin: sample.bin &C300
10 CALL &C300
```

YASM61860 も-r で.bin を出力しますが、出力されるのは番地 0 から始まる中身で、ORG で指定した番地より前は 0 で埋めてあります。YASM61860 の.bin を# bin: で読み込むと、ORG より前にある RAM(BASIC のプログラムや変数の場所など) まで 0 で書きつぶしてしまいます。YASM61860 でアセンブルしたプログラムは、.bin ではなく 16 進ダンプ (-d) を.hex として使ってください。

■指示の行　#で始まる行には、次の指示を書けます。.bas と.hex のどちらに書いてもかまいません。

| 書き方 | 意味 |
|---|---|
| `# title: 名前` | 一覧に出す名前。書かなければファイル名。 |
| `# run: 文字列` | 一覧から選んだときや`--run`で実行するときに、RUN モードで打つ文字列。書かなければ`RUN`。いくつも書くと上から順に打ち、2 つ目からは、前の命令が終わって入力待ちに戻るのを待ってから打つ。 |
| `# mode: PRO` | `# run:`の文字列を PRO モードで打つ。PRO モードで動かすマシン語のため。 |
| `# help: 説明` | 読み込んで動かし始めたときに、画面の下に 20 秒ほど出す説明 (遊び方など)。いくつも書くと行を重ねて出す。操作欄の「プログラム」のメニューの「読み込んだプログラムの説明」でもう一度出せる。一覧では、選んだ行の下に出る。 |
| `# setup: 命令` | `NEW`のあと、BASIC を打ち込む前に PRO モードで打つ命令。これがあるときは、ダンプも`NEW`とこの命令のあとで書き込む。`EQU#`で BASIC の置き場所を動かしてから、空いたところにマシン語を置く記事のためのもの。 |
| `# hex: ファイル名` | 先に書き込む 16 進ダンプ。同じフォルダ、置き場所の同じ機種のフォルダ、置き場所の直下の順に探す。1 つのマシン語を、いくつもの BASIC のプログラムから使える。 |
| `# bin: ファイル名 番地` | 先に書き込むマシン語のバイナリ (`.bin`) のファイル名と、置く番地。ファイルを探す場所と順番は`# hex:`と同じ。 |
| `# after: ファイル名` | 実行の文字列を打ったあとで書き込む 16 進ダンプ。BASIC で`DIM`などをして場所を空けてからマシン語を読み込む記事のためのもの。`.bin`を書き込むときは、`# after: ファイル名.bin 番地`のように番地も書く。 |
| `# data: 範囲` | 命令ではなくデータの範囲 (6.6 節)。`C100-C17F`のように始めと終わり (終わりの番地も含む) を書き、いくつもあれば`,`で区切る。終わりを書かずに`C100`とすると、そこから次の指定か実行した命令の手前までをデータにする。マシン語の作業欄の逆アセンブルで`DB`の行にする。 |
| `# model: 機種` | プログラムが動く機種 (`1251`、`1245`、`1261`など。いくつもあるときは`,`で区切る。`1261`と書いたものは PC-1260 と PC-1262 でも動く)。ほかの機種で動かしているときは一覧に出さず、ファイルを開いても「(PC-1251 用)」のように知らせて読み込まない。同じ`title`のものがいまの機種のフォルダにあれば、そちらだけが一覧に出る。書かなければ置いたフォルダで決まる (直下なら番地から考え、番地がなければどの機種でも)。いくつかの機種で動くものは直下に置いてこの行を書く。 |
| `# norom: 説明` | ROM なしで読み込んだとき (6.1 節) に、画面の下に出す説明。ROM なしでの遊び方の違いなどを書く。いくつも書くと行を重ねて出す。`# norom: off`と書くと、ROM なしの一覧に出さない。 |

`# hex:`や`# after:`で使われるだけの`.hex`(同じ名前の`.bas`がないもの) は、一覧には出しません。`.bin`のファイルは、どれも一覧には出ません。ダンプの大きさは、使う側のプログラムの行に出ます。

読めない行や、見つからない・読めないファイル、RAM の外に書くダンプ、RAM に入らない大きさの BASIC、打ち込んでも欠けてしまう長い行があるプログラムは、読み込もうとしても何も書き込まず、画面の下にその行やファイル、番地を出します。打ち込みで入れた BASIC が途中で RAM に入りきらなくなったときや、入った行の中身がファイルと違うときは、そのあとの実行の文字列と`# after:`の書き込みをやめます。PC-1350 で RAM カードを差しているときは、カードの番地に書くダンプも読み込めます。一覧では、読めない行のあるものに「読めない行あり」と出ます。

**読み込みの速さ**

BASIC は打ち込まずに、ROM が打ち込みで作るのと同じ中間コード (空白を落とし、命令の綴りを 1 バイトにしたもの) を RAM に直に置くので、キーの受け付けが遅い PC-1261 や PC-1350 でも、長いプログラムがすぐに入ります。中身は打ち込んだときとバイト単位で同じです。打つと 79 字を超える行も、行を呼び出して打ち足す手順 (5 節の BASIC のファイルの説明) で欠けずに入るものなら、そのまま置きます。貼り付け (command / Ctrl + V) は、中間コードにせずにキーを打って入れます。

**NEW で前のプログラムは消える**

`.bas` を読み込む前に `NEW` するので、入っていたプログラムは消えます。残しておきたいプログラムは、先に 5.9 節の方法でファイルに書き出しておいてください。
読み込む直前の状態 (RAM、CPU、スイッチ) は、エミュレータが終わるまで取っておきます。読み込んでから前のプログラムが要るとわかったときや、読み込みを途中でやめたときは、操作欄の「本体」のメニューの「プログラムを読み込む前に戻す」で元に戻せます (項目の下に、どのプログラムを読み込む前かが出ます)。書き出しに使うプログラムの名前や`# run:` なども、読み込む前のものに戻ります。取っておくのは最後に読み込む直前の 1 つだけで、別のプログラムを読み込むと入れ替わります。

## 5.3　端末から読み込む

起動するときにプログラムの名前かファイルの場所を書くと、起動してすぐに読み込みます。名前だけなら置き場所から探します。`--run` をつけると、読み込んだあと実行します。

```
$ uv run pc1251 primes --run
$ uv run pc1251 ~/basic/game.bas
```

Windows なら、ファイルの場所は `C:\Users\名前\basic\game.bas` のように書きます。

## 5.4　マシン語を動かす

マシン語は 16 進ダンプのファイルにして RAM に書き込み、BASIC の `CALL` で呼びます。PC-1251 の RAM は `B800`〜`C7FF` 番地の 4KB で、BASIC のプログラムは下の番地から、変数は上の番地から使います。マシン語は、BASIC のプログラムや変数とぶつからない番地に置いてください。マシン語を打ち込んで 1 命令ずつ調べるときは、マシン語の作業欄 (6 節) を使います。

## 5.5　CALL で表示をつけたまま計算する

BASIC はプログラムを実行しているあいだ、液晶の表示を切っています (C ポートのビット 0 を 0 にする)。マシン語でビット 0 を 1 にしてから計算を続けると、実行中も表示が消えません。見本の `noise.bas` は、表示をつけるマシン語 5 バイトを `POKE` で書いて `CALL` で呼び、計算しながら液晶に棒を描きます。

```
10 POKE &C200,2,1,255,177,55
20 CALL &C200
```

```
30 FOR I=0 TO 47
40 B$=STR$ (I*I)
50 POKE &F87B-I,127
60 NEXT I
70 WAIT 128:PRINT "DONE"
80 END
```

10 行で書くマシン語は次の 3 命令です。1FB1 番地は CPU の内部 ROM にあるルーチンで、A の値を C ポートに出します。

```
C200  02 01    LIA 01      A=&01(表示をオン)
C202  FF B1    CAL 1FB1    C ポートに A を出す
C204  37       RTN         BASIC に戻る
```

50 行は、液晶 RAM に直接 1 列ずつ点を書きます。液晶の右半分は F87B 番地が左端 (60 列目) で、番地が小さくなるほど右へ進むので、&F87B-I で棒が右へ伸びます。




表示をつけたままにすると、左の数桁に不規則な点の模様が出ます。ROM が液晶 RAM の一部を計算の作業場所に使っているためです。40 行の STR$は数を文字にする途中で F800〜F822 番地 (左から 7 桁ほど) を書き換え、ほかの計算でも右端の F840〜F847 番地などを使います。ふだんは実行中に表示を切っているので、作業場所の書き換えは目に見えません。

**CALL するマシン語の置き場所**

C200 番地は PC-インタープリタ (5.7 節) の置き場所の中にあり、C000 番地から置くマシン語の見本とも重なります。続けて使うときは、あとから読み込んだ方が残ります。

## 5.6　マシン語で液晶に描く

■文字を 1 ドットずつ流す　見本の scroll.hex と scroll.bas は、POCKET COMPUTER と機種名を、液晶の全幅で右から左へ 1 列ずつ流すマシン語です。4.4 節のように PRINT で流すと 1 文字 (5 列) ずつ跳びますが、液晶 RAM に直接書けば 1 列ずつ動かせます。文字の形はマシン語の後ろの表に持っていて、1 コマごとに表を読み始める場所を 1 列ずらし、液晶の幅ぶんを液晶 RAM へ写します。液晶は点が消えるのが遅く、細い線は流すとにじむので、線の太さ 2 列の太い文字にしてあります。液晶の 5 列ごとのすき間で文字が切れて見えるのは、実機でも同じです。.hex には 1 行ごとに ; のあとへニーモニックを書いてあります。同じ名前の scroll.bas が CALL で呼ぶので、一覧から選ぶと両方が読み込まれます。Esc (BRK) を押すと BASIC に戻ります。PC-1251 用は C000 番

地から、PC-1245 用は C100 番地から置きます。液晶の幅が違うので、2 つは別々のマシン語です。

■弾むボール　見本の ball.hex と ball.bas は、ボールが地面でひしゃげ、放物線を描いて弾みながら、液晶の全幅を右へ進み、右端まで行くと左端から出てくるマシン語です。1 回の弾みを 9 枚の絵にして表に持ち、1 枚ずつ液晶 RAM へ写します。着地の直前と跳ね上がった直後は縦に伸び、地面では横にひしゃげます。着地の前後の 3 枚は同じ桁に置きます。縦に伸びた絵は、着地の直前を 1 列左に、跳ね上がった直後を 1 列右に寄せ、ボールが斜めに入って斜めに出るようにしてあります。液晶は 5 列ごとに桁のすき間があり、ボールがすき間をまたぐと割れて見えるので、どの絵も 1 桁の枠に収めます。液晶の右半分は番地が逆向きに並ぶので、絵を描く桁が右半分にあるときは番地を減らしながら書きます。PC-1251 用は C000 番地から、PC-1245 用は C100 番地から置き、違うのは液晶の幅だけです。PC-1261 用は、2 行の液晶を使う別のマシン語です (9 節)。

■ブロック崩し　見本の pc1251/break.hex と break.bas は、横向きのブロック崩しです。左端のパドルを ↑ ↓ で動かし、Space か Enter で玉を打ち出して、右のブロックの壁を削ります。壁と画面の右端のあいだは空いているので、空いたところへ玉を回り込ませると壁を裏から削れます。壁はゆっくり左へ寄ってきて、先端が左から 12 列目まで来ると押し切られ、玉を 1 個失います。玉は 3 個で、全部なくなると BASIC に戻って得点を出します。右端の 3 つの点が残りの玉、残りの玉の右の 5 桁が得点です。

始めに 1:EASY 2:NORMAL ?と聞いてくるので、1 か 2 を押して ENTER を押します。EASY はパドルが縦 3 段で、玉は 1 コマに 1.5 列、壁は 60 コマで 1 列と遅めです。NORMAL はパドルが縦 2 段で、玉は 1 コマに 2 列、壁は 40 コマで 1 列です。ENTER だけを押すと前と同じ難易度で始まります。最高点は難易度ごとに覚えています。




パドルに当てた場所で玉の角度が変わり、パドルを動かしながら当てるとさらに角度がつきます。縦が 7 ドットしかないので、同じ角度のまま打ち返していると、同じ段ばかりを往復して最後の数個が残りがちです。角度を変えて狙ってください。面をクリアすると 500 点が入り、次の面から玉が少し速くなります。

面ごとに 1 つ、点滅しているブロックがあります。点滅しているブロックを壊すと、壊した場所からもう 1 個の玉が出て、2 個で遊べます。玉は 2 個までです。新しい玉は壊した玉と同じ向きに進み、壊した玉のほうが跳ね返るので、2 個はブロックの場所から別々の向きへ進みます。片方を落としても残りの玉は減らず、2 個とも落とすと 1 個失います。

マシン語は C000〜C56E 番地の約 1.4 KB(表は C501 番地から) で、液晶に描く前の絵を BF88 番地からの 120 バイトに組み立ててから、液晶の RAM へまとめて写します。2 個目の玉の位置と向きは BF84 番地からの 4 バイトに置きます。BASIC のプログラムは BF83 番地までに収まっている必

要があります (break.bas は B920 番地で終わります)。ROM のルーチンは呼ばず、キーは IA・IB のポートを直接読み、音は C ポートのビット 4〜6 を切り替えて出します。内部 RAM の 10h〜2Fh を変数に使うので、入るときに BASIC の値を C5E0 番地からの 32 バイトへ退避し、出るときに入れ替えて戻します。終わったあとも変数は退避先に残るので、得点の 5 桁は C5F6〜C5FA 番地から PEEK で読めます。break.bas は退避先の得点を読んで、得点と最高点を出します。難易度は、CALL の前に C5DF 番地へ POKE しておきます (1 なら EASY)。PC-1245 用 (8 節) も、得点の退避先と難易度の番地は同じです。

## 5.7　PC-インタープリタともぐらたたき

見本には、PiO 1986 年 8 月号の「PC-インタープリタ」(Fan_PC-1251 氏作) が入っています。BASIC のエディタで書いた行を独自の文法で読んで実行するマシン語のプログラムで、数はすべて 16 進法 2 桁の 1 バイト、命令は PRINT、IF〜NOT、GOTO、FOR〜NEXT、BEEP、WAIT、液晶に 1 列ずつ点を打つ DEG などです。

pcint.hex と pcint.bas

記事のリスト 1(C200〜C5CF の 976 バイト) と、PC-インタープリタを試す短いプログラムの組です。一覧から選ぶと、CALL &C300 で動き出します。

mogura.bas

記事のリスト 2 のもぐらたたきです。# hex: pcint.hex の行で PC-インタープリタを先に書き込み、RUN 900 で始めます。




もぐらたたきを始めるには、command / Ctrl + O (または操作欄の「プログラム」のメニューの「プログラムの一覧」) で開く一覧から「もぐらたたき (PC-インタープリタ)」を選びます。PC-インタープリタのダンプとリスト 2 が読み込まれ、RUN モードで RUN 900 が打たれてゲームが始まります。一度読み込んだあとは、RUN モードで RUN 900 と打つか、DEF = を押せば何度でも遊べます (900 行の頭に"="の印がある)。

900 行は CALL &C300 で PC-インタープリタを呼び、ゲームが終わって BASIC に戻ってから PRINT で得点を出します。CALL &C300 とだけ打っても遊べますが、得点は出ません。

遊び方は次のとおりです。液晶に 1〜6 の穴が並び、どこかの穴からもぐらが 3 段階で顔を出します。出た穴の数字キーを押しっぱなしにしておくと、当たりの音が鳴って得点が 1 つ増え、次のもぐらが出ます。当てるたびに少しずつ速くなります。3 段目まで出きっても押せないと GAME OVER になり、ENTER を押すと得点が出ます。

**記事のプログラムの扱い**

`pcint.hex` と `mogura.bas` は記事の作者 (Fan_PC-1251 氏) の著作物で、作者の了解を得て収めています。PC-インタープリタを 1 命令ずつ解説した『PC-インタープリタを読む』(おりぐち) も `doc/pcinterp.pdf` にあります。

## 5.8　ほかの読み込み方

command+V (Windows は Ctrl+V)
　クリップボードの文字を、そのままキーとして打ち込みます。`NEW` もモードの切り替えもしないので、打ち込む前に PRO モードにしておきます。

`--type ファイル`
　起動したあと、ファイルの中身をそのままキーとして打ち込みます。

`--load ファイル`
　起動したあと、16 進ダンプを RAM に書き込みます。

## 5.9　プログラムをファイルに書き出す

　エミュレータで打ち込んだり直したりした BASIC のプログラムは、ファイルに書き出して残せます。command/Ctrl+E を押すか、操作欄の「プログラム」のメニューで「プログラムをファイルに書き出す」を選ぶと、いまのプログラムをプログラムの置き場所 (`~/.pc1251/programs`) に `.bas` のファイルとして書き出します。ファイル名は「名前-日時.bas」で、名前は最後に読み込んだプログラムのもの (なければ `program`) です。最後に読み込んだものが機種ごとのフォルダにあったときは、置き場所の同じ名前のフォルダ (`pc1245` など) に書き出します。書き出したファイルはすぐに一覧に出るので、そのまま読み込み直せます。

　RAM のプログラムは中間コードになっているので、ROM の中にある命令の表を引いて文字に戻します。空白は ROM が打ち込みのときに捨てているので、命令の前後に読みやすいように補います。打ち込み直せば、元と同じ中間コードになります。

　読み込んだプログラムに組になる `.hex` があったときは、書き出したファイルの頭に `# hex:` の行を入れて、元のダンプを指しておきます。`# bin:` や `# after:` で `.bin` を読み込んでいたときは、同じ `# bin:`・`# after:` の行 (番地つき) を頭に入れておきます。`POKE` で書いたマシン語など RAM の中身そのものは書き出しません。

　エミュレータを起動しないで、保存してある RAM から書き出すこともできます。

```
$ uv run pc1251 --export ~/Desktop
```

## 5.10　プリンタに印字する

　エミュレータには、プリンタ (PC-1251・PC-1245 では CE-125、PC-1261・PC-1350 では CE-126P にあたるもの) がいつもつながっています。どちらも感熱紙に 5×7 の点で字を打つプリンタです。

LPRINT や LLIST で印字すると、本体の絵の下に紙が出てきて、新しい行が出るたびに紙が送られます。




- 紙には最後の 5 行が見えます。紙の上でホイールを回すと、前の行を見られます。
- 印字は紙の絵として、プログラムの置き場所に print-日時.png の名前で自動で保存します。印字が 1 秒とぎれるたびに書き直し、エミュレータを終えるときにも保存します。同じ名前のファイルがあるときは、print-日時-2.png のように番号を付けます。保存先は紙の左に出ます。保存先をクリックするか、メニューの「印字の保存先を開く」を選ぶと、そのフォルダを開きます。
- 紙の右の × か、操作欄の「表示・入力」のメニューの「プリンタの紙をしまう」で紙をしまいます。しまうとその紙は終わりで、次に印字すると新しい紙 (新しいファイル) になります。
- PC-1251・PC-1245 で SHIFT ENTER (P ↔ NP) を押して液晶に P を出しておくと、手で打った計算の式と答えも印字されます。

字の形は、プリンタのものではなく本体の ROM の字の形で代わりにしています。印字の速さと、紙送りのキーは再現していません。

## 5.11　カセットの音 (wav) で実機とやり取りする

実機のカセットインターフェースで読み書きする音を、wav のファイルにして実機とやり取りできます。

■エミュレータから実機へ　RUN モードで CSAVE か CSAVE M を打つと、ROM がカセットに出す音を、エミュレータがそのまま wav のファイルに書き出します。ファイルはプログラムの置き場所 (~/.pc1251/programs) に「tape-日時.wav」の名前で置かれ、画面の下にファイルの名前が出ま

す。一覧で O を押すと、置き場所のフォルダを開けます。実機に読み込ませるときは、wav をパソコンで再生し、実機のカセットインターフェースの入力につないで、実機で CLOAD(マシン語は CLOAD M) を打ちます。

```
CSAVE "BREAK"
CSAVE M "BREAK";&C000,&C56E
```

エミュレータは、4kHz と 2kHz の音を切り替えるカセットの信号が 2 秒以上続いたときだけ、音が止まって 1 秒たったところで wav を書き出します。BEEP の音は書き出しません。前置きの音が 8 秒あるので、書き出しまでには、どんなに短いプログラムでも 10 秒ほどかかります (command/Ctrl+T で速くすると短くなります)。

■実機からエミュレータへ　実機で CSAVE した音をパソコンで録音して wav にし、ウィンドウにドロップすると、テープをセットした状態になります (起動のときに--tape ファイルを付けても同じ)。テープをセットしたあと、RUN モードで CLOAD か CLOAD M を打つと、ROM がテープを読み始めたところで再生が始まり、実機と同じ時間をかけて読み込みます (前置きの音だけで 8 秒、ブロック崩しのマシン語は 1 分ほど。command/Ctrl+T で速くもできます)。

実機用のテープの音をエミュレータで読み込めることは、PC-1251 の見本プログラムが入ったマイクロカセットを録音した wav(20 本) で確かめました。20 本とも CLOAD でエラーなく読めます。PocketTools の bin2wav や、YASM61860 の-w(-old をつけたとき) で作ったマシン語の wav も、PC-1251 と PC-1245 のどちらでも CLOAD M で読めます。

**書き出した wav を実機で読み込ませてはいません**

エミュレータの CSAVE で書き出した wav を実機で読み込ませることは、まだ確かめていません。

## 6　マシン語の作業欄

マシン語の作業欄は、本体の絵の右に出る欄です。マシン語を 16 進ダンプで打ち込み、実行し、1 命令ずつ進めて調べられます。ROM のファイルがなくても使えるので、実機から ROM を読み出せない人も、マシン語のプログラムを書いて試せます。ROM があるときは、メニューで作業欄を出して、BASIC の CALL で呼んだマシン語を調べるのに使えます (6.8 節)。




## 6.1　ROM なしで起動する

使う機種の ROM のファイルが見つからないとき (大きさが足りないときも) は、ROM なしで起動して、作業欄を出します。ROM を置いてあっても ROM なしで使いたいときは、`--norom` をつけて起動します。PC-1350 は ROM なしでは起動できません。

```
$ uv run pc1251 --norom
$ uv run pc1251 --model 1245 --norom
```

ROM なしでも、CPU、液晶、キー、ブザー、タイマは、ROM があるときと同じように使えます。BASIC はないので、液晶に>は出ず、マシン語は作業欄から実行します。ROM のルーチンもないので、マシン語が ROM の番地を呼ぶと、そこで止まります (6.5 節)。ただし、次の 2 つのルーチンは、エミュレータが働きをまねるので、ROM なしでも呼べます。

| 番地 | 働き |
|---|---|
| `1F44` | キーを 1 回読む。キーが押されていればＣを 1 にして A にキーの文字コードを入れ、押されていなければＣを 0 にする。 |
| `1FBC` | A に入れた文字コードの字の形 (5 バイト) の番地から 1 を引いた値を、X に入れる。 |

字の形の表は、PC-1251 の ROM と同じ番地と並びで置いてあるので、ROM の字の形の表を読むマシン語もそのまま使えます。字の形はエミュレータで描いたもので、ROM の字の形とは細かいところが違います。

ROM なしで使った RAM は、ROM で動かしたときの RAM とは別に保存します (ホームの`.pc1251` フォルダの `ram-1251-norom.bin` など)。終わるときには、RAM と一緒に、開始番

地、止める所、作業欄から書いた範囲、逆アセンブルで命令とデータを見分けるのに使う情報 (実行した番地と、自分で決めた指定。6.6 節) も保存し、次に起動したときに元に戻します。どこでもセーブ (3.5 節) の保存先も、ROM なしと ROM ありで別になります。

■ROM なしでプログラムを読み込む　ROM なしで command/Ctrl+O の一覧を開くと、マシン語のダンプがあるプログラムだけが並びます。読み込むと、ダンプを RAM に書き、組になる BASIC の POKE のうち、値を数だけで書いたものも書きます (IF のあとの POKE は、条件しだいなので書きません)。BASIC そのものは実行しません。開始番地は、BASIC の最後の CALL の番地になります (CALL がなければ最初のダンプの番地)。読み込んだら command/Ctrl+G で実行します。見本の弾むボールとブロック崩しは、ROM なしでも遊べます。PC-インタープリタのように、ROM なしでは使えない見本には# norom: off の行を書いてあるので、一覧に出ません。# norom: off の行がないプログラムは、ROM のルーチンを呼ぶものでも一覧に出て、呼んだところで止まります。

## 6.2　作業欄の各部

見出しとボタン

いちばん上に「マシン語 (ROM なし)」と、いまの状態 (編集中、実行中、一時停止中) が出ます。その下のボタンは、状態に合わせて入れ替わります。ボタンにマウスを載せると、押したときに何をするか (次に実行する命令など) と、ショートカットが出ます。

開始番地

実行を始める番地です。クリックすると、16 進表記の 4 桁を打って変えられます (6.5 節)。自分で決めるまでは、横に「(自動)」と出ます。右の「実行の前に液晶を消す」に印がついていると、実行の前に液晶を消して、表示をつけます。

書き出し

「.hex に書き出す」のボタンと、書き出す範囲のバイト数と番地が出ます (6.7 節)。同じ行の右端に、書き込みを取り消す・やり直すボタンがあります (6.4 節)。

ダンプ

RAM の中身を 1 行に 8 バイトずつ、番地、+0〜+7 のバイト、Σ (チェックサム) の順に並べます (6.3 節)。カーソルのある命令のバイトには帯がつき、開始番地のバイトの左には▶の印がつきます。実行を始めてから書き換わったバイトは色を変え、書き込めない番地のバイトは暗く出します。

逆アセンブル

カーソルのある命令のまわりを、命令ごとに番地、バイト、ニーモニックの順に出します。データとみたところは DB の行にして、色を変えて出します (6.6 節)。行の右端の印 ( ✓は実行済み、. は静的解析、?は推定、U はユーザ指定) は、命令かデータかをどう見分けたかを表し、逆アセンブルの下に印の見方が出ます。行の左の丸をクリックすると、その命令に止める所を置きます (もう一度クリックすると外します)。行をクリックすると、ダンプのカーソルがその命令へ移ります。開始番地の命令の行には、▶の印と「開始」が出ます。

レジスタ

I、J、A、B、K、L、M、N、X、Y、DP、PC、P、Q、R、キャリー (C) とゼロ (Z) のフラグ、実行を始めてからのサイクル数です。前に止まったときから変わったレジスタは色を変えます。

内部 RAM

CPU の内部 RAM(00〜5F 番地) を 16 バイトずつ出します。00〜0B 番地は I、J、A、B、XL、XH、YL、YH、K、L、M、N です。5C〜5F 番地は入出力のポートの値で、暗く出します。スタックの頭 (R の指す番地) には帯がつきます。見出しの「内部 RAM」をクリックすると、行を開く・閉じるを切り替えます。

知らせ

いちばん下に、止まった理由や、操作の結果が出ます。

## 6.3　ダンプリストの読み方

本や雑誌のマシン語のプログラムは、ダンプリストの形で載っていることがよくあります。ダンプリストの 1 行は、番地、バイトを 16 進表記で並べたもの、チェックサムでできています。

```
C000 027F10F8005237 12
```

番地の C000 は、その行の最初のバイトを置く番地です。続く 027F10F8005237 は 02、7F、10、F8、00、52、37 の 7 バイトで、C000〜C006 番地に置きます。長いプログラムでは 1 行に 8 バイトずつ並び、行の番地は 8 ずつ増えます。最後の 12 がチェックサムで、行のバイトをすべて足した和 (212) の下 2 桁です。打ち込んだ行のチェックサムを計算してリストのチェックサムと比べると、打ち間違えた行を見つけやすくなります。ただし、バイトの順序の入れ替えのように、和が変わらない間違いはチェックサムでは見つかりません。チェックサムの計算のしかたや 1 行のバイト数は、本や雑誌によって違うことがあるので、記事の説明を確かめてください。

例の 7 バイトは、PC-1251 の液晶の左端の 1 列に縦の線を描いて戻るマシン語です (LIA 7F、LIDP F800、STD、RTN)。ROM のルーチンを呼ばないので、ROM なしでも打ち込んで実行できます。

作業欄のダンプも 1 行に 8 バイトを並べ、右端の Σ の列に、その行の 8 バイトの和の下 2 桁を出します。ダンプの行は開始番地から 8 バイトずつ区切るので、ダンプリストの最初の番地から打ち込めば、行の区切りもダンプリストとそろいます。打ち込んだ行の Σ がリストのチェックサムと違えば、その行に打ち間違いがあります。リストの最後の行のように 8 バイトに足りない行では、残りのバイトも和に入るので、残りのバイトが 00 でなければ、Σ はリストのチェックサムと合いません。

## 6.4　ダンプを打ち込む

編集中は、パソコンのキーで作業欄のダンプに書き込みます。ダンプリストは、次の順に打ちます。

1. 番地の 4 桁を打ちます。打ち終えると、カーソルがその番地の行のバイトの欄へ移ります。開始番地がまだ決まっていなければ、開始番地もその番地になります。別の番地を打ち直すときは、

Tabで番地の欄へ移ります (カーソルが移るだけで、開始番地は変わりません)。

2. バイトを 2 桁ずつ打ちます。番地のすぐあとの空白は、打っても読み飛ばします。1 行の 8 バイトを打ち終えたら、Σがリストのチェックサムと同じか確かめます。チェックサムは打ちません。
3. Enterで次の行の最初のバイトへ移ります。行の番地は自動で 8 つ先になるので、2 行目からはバイトだけを打ちます。

| キー | すること |
|---|---|
| 0〜9、A〜F | カーソルの所に 16 進表記の 1 字を書く |
| Enter | 次の行の最初のバイトへ移る |
| Space | 番地の欄からバイトの欄へ移る。バイトの欄では次のバイトへ移る |
| Tab | 番地の欄の最初の桁へ移る (番地を打ち直す) |
| ←　→ | 1 字ずつ戻る・進む。行の最後のバイトから進むと次の行の最初のバイトへ、行の最初のバイトから戻ると前の行の最後のバイトへ移る (Backspaceも 1 字戻るだけで、字は消さない) |
| ↑　↓ | 1 行上・下へ移る |
| PageUp　PageDown | 1 画面ぶん上・下へ移る |
| Home　End | 行の最初のバイト・行の終わりへ移る (command/Ctrl+←・command/Ctrl+→でも同じ)。行の最初のバイトでHomeを押すと番地の欄へ移る |
| command/Ctrl+↑　command/Ctrl+↓ | 開始番地の行・書いてあるところの終わりへ移る |
| command/Ctrl+Z | 直前の書き込み (1 バイトの打ち込みか、1 回の貼り付け) を取り消す |
| shift+command/Ctrl+Z | 取り消した書き込みをやり直す |

マウスでは、ダンプの番地やバイトをクリックするとカーソルがそこへ移り、作業欄の上でホイールを回すとダンプが 1 行ずつ動きます。

書き込みの取り消しとやり直しは、「.hex に書き出す」の行の右端にある「取り消す」「やり直す」のボタンと、操作欄の「プログラム」のメニューからもできます。ボタンにマウスを載せるか、メニューの項目の下を見ると、何を取り消すか (「C003 番地を AB から 00 に戻す」など) がわかります。取り消せないときは、ボタンと項目が暗くなります。

取り消せるのは作業欄で書き込んだもの (打ち込みと貼り付け) だけで、直近の 500 回までです。マシン語を実行して書き換わったメモリは戻りません。ROM なしで実行しているあいだは取り消せません。一覧からプログラムを読み込んだとき、状態を戻したとき、エミュレータを終わったときは、それまでの書き込みは取り消せなくなります。読み込みそのものは、「本体」のメニューの「プログラムを読み込む前に戻す」で戻せます (5 節)。

■貼り付ける　ダンプリストや.hex の行をクリップボードにコピーしてcommand/Ctrl+Vを押すと、まとめて書き込みます。番地のない行は、前の行の続きの番地に書きます (最初の行に番地がなければカーソルの所から)。チェックサムのついた行は、バイトの和とチェックサムを比べ、合わない行の番地を下の知らせに出します。RAM の外の番地には書きません。開始番地を自分で

決めていなければ、貼り付けた最初の行の番地が開始番地になります。.hex のファイルは、一覧 (command / Ctrl + O) から読み込んでもかまいません。

## 6.5　実行して調べる

| すること | ボタン | キー |
|---|---|---|
| 開始番地から実行する | ▶ 実行 | command / Ctrl + G |
| カーソルの番地を開始番地にする | | command / Ctrl + A |
| カーソルの番地を開始番地にして実行する | | shift + command / Ctrl + G |
| 実行を一時停止する | ■ 一時停止 | command / Ctrl + .（command / Ctrl + P でも同じ） |
| 一時停止したところから続ける | ▶ 続ける | command / Ctrl + G |
| 次の 1 命令だけ実行して止まる | 1 命令ずつ、1 命令 | command / Ctrl + J |
| 呼び出しの命令を、呼んだ先から戻るまで実行して止まる | CALL ごと | shift + command / Ctrl + J |
| カーソルの命令に止める所を置く・外す | ● 止める所 | command / Ctrl + B |
| 一時停止中に、実行をやめて編集に戻る | 編集へ | Esc |

**■開始番地を決める**　開始番地は、まだ決めていなければ自動で決まります。最初は C000 番地 (RAM が C000 番地を含まない機種では RAM の最初の番地) で、ダンプに最初に打った番地、貼り付けた最初の行の番地、一覧から読み込んだプログラムの CALL の番地になります。ダンプに書き始めたあとに番地を打っても、カーソルが移るだけで、開始番地は変わりません。開始番地は、次のどれかで自分で決めます。

- ダンプのカーソルを置いて command / Ctrl + A を押す。
- ダンプのバイトか逆アセンブルの行を右クリックし、「C010 番地を開始番地にする」のような項目を選ぶ。同じメニューで、その番地から実行することと、止める所を置く・外すこと、その番地からデータか命令に決めること (6.6 節) もできます。
- 「開始番地」の欄をクリックし、16 進表記の 4 桁を打つ。4 桁打つと決まります。Backspace で 1 字消し、Esc でやめます。command / Ctrl + V で番地 (C010 や&C010) を貼り付けても決まります。欄に打っているあいだは、command / Ctrl + V と command / Ctrl + Z は欄にだけ効き、ダンプは変わりません。

自分で決めた開始番地は、番地の打ち込みや貼り付け、書き込みの取り消しでは変わりません。一覧からプログラムを読み込むと、そのプログラムの開始番地に変わります。ROM なしでは、RAM の外の番地は開始番地にできません。

実行を始めると、エミュレータはスタックの頭 (R) を 5C にし、戻り先として 0000 番地を積んでから、開始番地へ飛びます。マシン語が最後に RTN で戻ると実行が終わり、編集中に戻ります。BASIC から CALL で呼ぶマシン語と同じく、最後は RTN で戻るように書いてください。

実行中は、パソコンのキーが本体のキーになります (Tab が SHIFT、option (Windows と Linux で

は Alt ) が DEF 、 Delete が CL 、 Esc が BRK )。キーを読むマシン語も、そのまま試せます。裏の RESET ボタン ( command / Ctrl + R 、 F5 ) を押すと、実行をやめて編集中に戻ります。

編集中に command / Ctrl + J を押すと、開始番地の最初の命令を実行する前で止まります。そこから command / Ctrl + J を押すたびに、1 命令ずつ進みます。次の命令が CAL や CALL のような呼び出しのときは、「CALL ごと」( shift + command / Ctrl + J ) で、呼んだ先が RTN で戻ってくるまで実行し、呼び出しの次の命令で止まれます。

止める所は、逆アセンブルの行の左の丸をクリックするか、ダンプのカーソルを命令に置いて command / Ctrl + B を押すか、ダンプや逆アセンブルの行を右クリックして置きます。実行がその命令に来ると、命令を実行する前で止まります。

止まったときは、ダンプのカーソルが次に実行する命令 (PC の指す命令) へ移ります。ダンプでは、実行を始めてから書き換わった RAM と液晶のバイトの色が変わり、レジスタと内部 RAM では、前に止まったときから変わった値の色が変わります。

マシン語が RAM の外の番地 (ROM の中のルーチンなど) へ飛ぶと、飛んだ先で止まり、下の知らせに番地と、わかれば ROM のルーチンの名前を出します。ROM なしではその先を実行できないので、「編集へ」で編集に戻ります。止める所も、RAM の外には置けません。

## 6.6　命令とデータを見分ける

マシン語のプログラムには、命令のあいだに文字や表や字の形などのデータが入っています。データを命令として読むと、意味のない命令が並ぶうえ、その先の命令の切れ目もずれます。作業欄の逆アセンブルは、RAM の番地ごとに、命令の頭か、命令の続きのバイトか、データかを見分けて、その切れ目で行を出します。データは DB の行にして、1 行に 4 バイトずつ、どのバイトも文字なら文字も添えて出します (DB 48 45 4C 4C 'HELL')。

見分け方は次の 4 つで、逆アセンブルの行の右端に、どれで決めたかの印が出ます。印にマウスを載せると、理由が短く出ます。

| 印 | 見分け方 |
|---|---|
| ✓ | 実行済み。作業欄を出しているあいだに、CPU がその番地から命令を読んで実行した。ROM なしではいつも、ROM のある本体では作業欄を出しているあいだと、1 命令ずつ進めたときに覚える |
| . | 静的解析。実行せずに、入口から分岐と呼び出しをたどって命令と判定した。入口は、開始番地、読み込んだプログラムの BASIC の CALL の番地、止める所、実行済みの命令。ジャンプと呼び出しの先、条件つきのジャンプでは飛ぶ先と次の命令の両方、呼び出しのあと (呼んだ先から戻るものとみる) をたどり、RTN と条件なしのジャンプのあとはたどらない |
| ? | 推定。どこからも参照されていないので、中身の形から推定した。文字が 5 字以上並んでいれば文字列、00 か FF が 4 バイト以上並んでいれば埋め草、残りは命令として読んでみる。RTN か条件なしのジャンプで終わる並びは命令とし、確かではないので暗く出す。ない命令か RAM と ROM の外へ飛ぶ命令に当たるところと、RTN か条件なしのジャンプで終わらないまま途切れるところ (02 01 03 02 のように、命令としては読めても終わりのないもの) はデータと推定する |
| U | ユーザ指定。右クリックで、データか命令に指定した (下の「ユーザ指定」) |

PTC と DTC の表は、PTC の件数から表の行の数を求め、表の 1 行 (値と行き先の 3 バイト) と、

どれでもないときの行き先 (2 バイト) を DB の行にして、行き先を添えます (DB 01,C1,23 ; → C123)。表の行き先と、PTC で決めた戻り先も、静的解析の入口になります。PTC が見つからないときは、DTC のあとも命令として読みます。

推定するのは、作業欄から書いた範囲 (書き出す範囲。6.7 節) の中だけです。範囲の外は、これまでどおりカーソルのまわりを命令として読みます。実行中のプログラムが RAM を書き換えると、見分けを作り直します (実行中は 0.3 秒に 1 回まで)。打ち込み、貼り付け、読み込みで書き換えた番地は、実行済みでなくなります。

ROM のある本体でも、作業欄をしまっているあいだは実行した番地を覚えません (BASIC の速さは変わりません)。マシン語を CALL で呼ぶ前に作業欄を出しておくと、実行した命令に ✓ の印がつきます。

■ユーザ指定　見分けが違っているときは、ダンプのバイトか逆アセンブルの行を右クリックして、次の項目を選びます。

C010 番地からデータにする

その番地から、次の指定か実行済みの命令の手前まで (書いた範囲の終わりまで) をデータに指定します。静的解析で命令と判定されるところでも、データとして扱います。

C010 番地から命令にする

その番地を命令の頭に指定し、そこから静的解析します。

C010 番地の指定を消す

指定をやめて、推定に戻します。

指定は、状態の保存 (3.5 節) とエミュレータの終わりに一緒に保存します。.hex に書き出すと、データの指定は# data: の行に入ります (6.7 節)。一覧からプログラムを読み込むと、読み込んだ範囲の指定は消え、ファイルに# data: の行があればその範囲がデータの指定になります。

## 6.7　打ち込んだマシン語をファイルに書き出す

作業欄で打ち込んだマシン語は、.hex のファイルに書き出して残せます。command/Ctrl+E か「.hex に書き出す」のボタンを押すと、置き場所の機種のフォルダ (~/.pc1251/programs/pc1251 など) に、dump-日時.hex の名前で書き出します。書き出すのは、作業欄から書いた番地 (打ち込み、貼り付け、一覧からの読み込みで書いたところ。00 を書いたバイトも含む) です。ROM なしの本体では、開始番地から、00 が 16 バイト続く手前までも書き出します。書いた番地どうしのあいだが 64 バイトより短ければ、書いていないあいだのバイトも含めて 1 つの範囲にします (C000 と C010 に 3 バイトずつ書けば、C000〜C012 の 19 バイト)。離れた範囲がいくつかあれば、全部を書き出します。書き出す範囲とバイト数は、「.hex に書き出す」のボタンの下に出ます。

書き出したファイルの各行には、番地とバイトのあとに、; に続けてニーモニックを書き添えます。データとみたところは、逆アセンブルと同じ DB の行になります (6.6 節。; のあとは読み込むときに読み飛ばすので、読み込んだ結果は変わりません)。頭には# run: CALL &開始番地の行が入るので、一覧から読み込むと、ROM なしでは開始番地が元のとおりになり、ROM のある本体ではその CALL で実行します。自分でデータに決めた範囲があれば、# data: C100-C17F のような行

も入り、読み込むと同じ範囲がデータになります。

```
; C000〜C006(マシン語の作業欄から書き出し)
# run: CALL &C000
C000 02 7F       ; LIA 7F
C002 10 F8 00    ; LIDP F800
C005 52          ; STD
C006 37          ; RTN
```

### 6.8　ROM のある本体で作業欄を使う

ROM があるときは、command/Ctrl+Dを押すか、操作欄の「表示・入力」のメニューで「マシン語の作業欄を出す」を選ぶと、本体の右に作業欄が出ます。もう一度command/Ctrl+Dを押すとしまいます。作業欄を出しているかどうかは、次に起動したときも同じになります。

ROM のある本体は、作業欄を出しても ROM で動き続けます。作業欄の見出しの横に「キーは本体へ」と出ているあいだは、パソコンのキーは本体に届きます。作業欄をクリックすると「キーは作業欄へ」に変わり、ダンプの打ち込みと作業欄のショートカット (command/Ctrl+E、command/Ctrl+V なども) が作業欄に届きます。Escを押すと、キーは本体に戻ります。

作業欄で打ち込んだマシン語は、BASIC の `CALL` で呼びます。開始番地の横に、`CALL &C000` のように打つ命令が出ます。開始番地の決め方は ROM なしと同じです (6.5 節)。ROM のある本体では、作業欄に実行のボタンはなく、液晶も消しません。

止める所を置いておくと、`CALL` で呼んだマシン語がその命令に来たときに本体の実行が一時停止し (command/Ctrl+Pで止めたときと同じ状態)、作業欄をしまっていれば作業欄が出ます。そこから「1 命令」や「CALL ごと」で進め、「▶ 続ける」で本体の実行を続けます。「■ 一時停止」を押すと、本体がそのとき実行していた命令 (ふだんは ROM の中の命令) の前で止まります。ROM のある本体では、ROM の中の命令にも止める所を置け、1 命令ずつ進められます。

## 7　PC-1250・PC-1255 として使う

PC-1250 と PC-1255 は、PC-1251 と同じ ROM で、違うのは RAM の範囲だけです。起動のときに`--model 1250` か`--model 1255` を付けると、その機種として動きます。ROM は PC-1251 の `cpu-1251.rom` と `bas-1251.rom` を使います。

| 機種 | RAM | BASIC のプログラムの先頭 | 使える大きさ (`MEM`) |
|---|---|---|---|
| PC-1250 | `C000`〜`C7FF`(2KB) | `C030` | 1438 バイト |
| PC-1251 | `B800`〜`C7FF`(4KB) | `B830` | 3486 バイト |
| PC-1255 | `A000`〜`C7FF`(10KB) | `A030` | 9630 バイト |

RAM の範囲は MAME の実装によります。ROM は起動のときに RAM の頭を下へ探し、書いた印が上の区画に写るかどうかで終わりを決めるので、PC-1255 では `8000`〜`9FFF` を `A000`〜`BFFF`

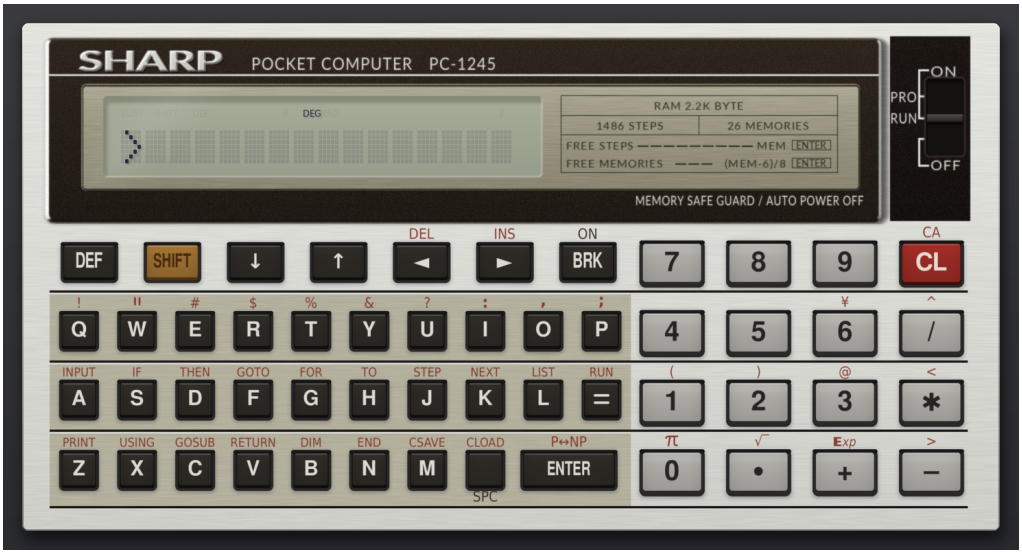
の鏡としました。鏡があると、MEM の値がカタログの 9630 バイトと合います。

筐体は PC-1251 の絵の型番だけを書き換えたものです。実機の筐体の色や刷り、細部の形の違いは再現していません。プログラムの一覧には、PC-1251 用のもののうち、書き込むマシン語の番地が RAM に収まり、BASIC のプログラムの置き場所 (見積もり) にかからないものが出ます。PC-1250 では BASIC が C030 番地から置かれるので、C000 番地から置くブロック崩しなどは出ません。終わるときの RAM は~/.pc1251/ram-1250.bin、ram-1255.bin に保存します。

# 8　PC-1245 として使う

PC-1245 は、PC-1250/1251 と同じ大きさの筐体です。CPU は同じ SC61860 ですが、ROM は PC-1251 とは別のものです。起動のときに--model 1245 を付けると、PC-1245 として動きます。

```
$ uv run pc1251 --model 1245
```




ROM は、実機から読み出したものを cpu-1245.rom と bas-1245.rom の名前で rom フォルダに置きます。読み出し方を書いたページも 2.4 節で紹介しています。

.bas や.hex のファイルを使う (5 節)、書き出す (5.9 節)、command / Ctrl + O などのショートカット (3.4 節)、DEF キーでの実行 (4.5 節) は、PC-1251 と同じように使えます。違うのは次のところです。

## 8.1　液晶

液晶は 16 桁です。PC-1251 と同じ液晶 RAM のうち、左の 12 桁 (F800〜F83B) と、右側の初めの 4 桁 (F87B〜F868) だけが見えます。17 桁目より右に書いたものは表示されないので、PC-1251 用のプログラムには表示が途中で切れるものがあります。

表示記号は BUSY、DEF、SHIFT、P、DE・G・RAD(DEG などの角度の単位)、E です。モード (PRO、RUN) は液晶に出ず、スイッチの位置で見ます。表示記号の位置は、実機の写真で見える DEG のほかは推定です。

液晶の右には、容量の表 (RAM 2.2K BYTE、1486 STEPS、26 MEMORIES) と、空きの調べ方が刷ってあります。MEM で残りのステップ数が、(MEM-6)/8 で残りの変数の数がわかります。

## 8.2　モードスイッチ

スイッチは PRO、RUN、OFF の 3 段です。PRO と RUN の 2 段は「ON」の括りに入っていて、電源が入る位置です。RSV モードはないので、4.6 節の文字列の登録は使えません。F3 は何もせず、command/Ctrl+↑・command/Ctrl+↓と操作欄の「本体」のメニューも、PRO、RUN、OFF の 3 段のあいだだけを動きます。

## 8.3　キー

キーの位置は PC-1251 と同じですが、刷ってある文字が違います。

- A〜=の段とZ〜SPCの段は、SHIFTを押してから打つと、キーの上に刷ってある BASIC の命令が入ります。たとえばSHIFT Zで PRINT、SHIFT Gで FOR、SHIFT =で RUN、SHIFT SPCで CLOAD です。
- 括弧はSHIFT 1が (、SHIFT 2が) です (PC-1251 は↓と↑)。パソコンで(、)を打つと、エミュレータがSHIFT 1、SHIFT 2に直して打ちます。貼り付けやファイルの読み込みでも同じです。
- ほかの記号は付録 B の表と同じです。

## 8.4　メモリと見本のプログラム

RAM は 2KB(C000〜C7FF) で、BASIC のプログラムは C000 番地から置かれます。プログラムに使えるのは 1486 ステップ (バイト) です。終わるときの RAM は、PC-1251 の ram.bin とは別の~/.pc1251/ram-1245.bin に保存します。

見本のプログラムは同じ一覧から読み込めますが、次の点に気をつけてください。

- ブロック崩し (5.6 節) は、programs/pc1245 にある PC-1245 用の break.hex と break.bas が読み込まれます。遊ぶ場所は 75 列で、右端の桁に残りの玉が出ます。遊んでいるあいだは得点を出さず、終わってから出します。点滅するブロックはなく、玉はいつも 1 個です。マシン語は C100〜C55D 番地で、BASIC のプログラムは C000〜C0FF に収まるようにしてあります。
- PC-インタープリタは C200〜C5CF に入ります。PiO の解説のとおり、PC-1245 で残るフリーエリアは 510 バイトです。
- ほかの見本は動きますが、24 桁を使う表示は 17 桁目から右が見えません。

# 9　PC-1261 として使う

PC-1261 は、2 行 ×24 桁の液晶を持つ機種です。CPU は同じ SC61860 ですが、ROM は PC-1251 とは別のもので、BASIC の命令も少し違います。起動のときに`--model 1261` を付けると、PC-1261 として動きます。

```
$ uv run pc1251 --model 1261
```




ROM は、実機から読み出したものを `cpu-1260.rom` と `bas-1260.rom` の名前で `rom` フォルダに置きます。PC-1260 と PC-1262 は PC-1261 と同じ ROM で、違うのは RAM の大きさだけです。`--model 1260`、`--model 1262` としても、同じ ROM のファイルを使います。筐体の絵は PC-1261 のものの型番だけを書き換えたもので、実機の筐体の色や刷り、細部の形の違いは再現していません。

`.bas` や `.hex` のファイルを使う (5 節)、書き出す (5.9 節)、command / Ctrl + O などのショートカット (3.4 節)、DEF キーでの実行 (4.5 節)、カセットの音 (5.11 節) は、PC-1251 と同じように使えます。違うのは次のところです。

## 9.1　液晶

液晶は 2 行 ×24 桁です。液晶の下の金属の板に `BUSY`、`PRINT`、`DEG`、`RAD`、`ERROR` が刷ってあり、そのすぐ上に液晶の短い横棒が点きます。液晶の左端には、上からカナ、`SMALL`、`SHIFT`、`DEF` の記号が並んでいます。カナの入力 (SHIFT 8) のあいだはカナが点きます。`SMALL` は英小文字の入力の記号です。RUN モードで `POKE &203C,PEEK &203C AND &7F` と打つと、SHIFT 8 がカナの代わりに英小文字の入力を切り替えるようになり、そのあいだ `SMALL` が点きます。電源を入れ直す

と、SHIFT 8はカナの切り替えに戻ります。角度の単位を GRAD にしたときに点く記号は、液晶にありません。

## 9.2 キー

キーの並びは PC-1251 と同じで、刷ってある文字が違います。

- 括弧はSHIFT 1が (、SHIFT 2が)、¥はSHIFT 6です。パソコンで(、)、¥を打つと、エミュレータがそのキー操作に直して打ちます。
- Exp のキーはありません。指数は英字の E で書きます (1E3 など)。
- SHIFT 7(HELP) で、BASIC の命令の一覧が出ます。
- SHIFT 8(カナ) で、カナの入力に切り替わります。もう一度押すと英字の入力に戻ります。カナの入力のあいだは、各キーの右下に刷ってあるカナが入り、SHIFTを押してから打つと右上のカナが入ります。「・」の印のあるキーは、SHIFTを押してから打つと小さい字 (ァ、ッ、ャなど) になります。数字の段の右に刷ってある゛゜。、「」・ーも、カナの入力のときに打てます。

パソコンでカナ (全角のカタカナ、半角のカタカナ、ひらがな) を打つか貼り付けると、エミュレータがカナの入力に切り替えて打ち、打ち終わったら英字の入力に戻します。「ガ」のような濁点のついた字は、「カ」のあとに゛を打つ形に直します。ファイルの読み込みでも同じです。

PC-1261 の ROM は、キーを離してから次のキーを受け取れるようになるまでの時間が PC-1251 より長いので、貼り付けでは 1 打鍵に 0.22 秒ほどかけます。パソコンのキーで打つときは、ROM の入力待ちに届けるあいだだけ速く動かすので、遅れません。ファイルから読み込む BASIC のプログラムは、ほかの機種と同じく中間コードを RAM に直に置くので、すぐに入ります。

## 9.3 メモリと見本のプログラム

RAM は 10KB(4000〜67FF) で、BASIC のプログラムは 4080 番地から置かれます。プログラムに使えるのは 9342 バイト (MEM の値) です。PC-1260 は RAM が 4KB(5800〜67FF) で、使えるのは 3198 バイトです。終わるときの RAM は~/.pc1251/ram-1261.bin(PC-1260 は ram-1260.bin、PC-1262 は ram-1262.bin) に保存します。角度の単位などの表示記号の状態を戻すため、液晶の RAM も一緒に保存します。

見本のプログラムのうち、PC-1251 の番地を使うマシン語 (PC-インタープリタ、もぐらたたき、noise.bas) は、一覧に出ません。PC-1261 用の見本は programs/pc1261 にあります。ball.hex と ball.bas は、ボールが 2 行目の下端で弾んで 1 行目の上端まで上がり、同じ高さで弾み続けながら右へ進むマシン語です。右端から出た分は左端から出てきます (5A00〜6142 番地。CALL &5A00 で動かし、Esc(BRK) で BASIC に戻ります)。位置は重力の式で 1/30 秒ごとに計算し、速く動いているときは縦に伸び、地面ではつぶれて横に広がり、いちばんつぶれたところでしばらく止まってから跳ね上がります。地面に着いているあいだだけ、着地で低い「ペタ」、跳ね上がるときに「びよん」と小さく鳴ります。音は、液晶を書き換える合間に C ポートのビット 4 を上げ下げして出し、ビットを立てておく時間を短くして小さくしています。ボールの形は、液晶のドットの

本当の位置 (桁と行の区切りのすき間を含む) から決めてあるので、すき間にかかると欠けて見えます。`hello.bas` は `CURSOR` で表示の位置を決めて、2 行の液晶にカナと記号を出します。`pc1261` のフォルダに置いたものは、PC-1260 と PC-1262 でも、マシン語の番地が RAM に収まり、BASIC の置き場所にかからなければ使えます。PC-1260 は RAM が狭いので、コンパイラ LC-3 とその見本は使えません。

■コンパイラ LC-3　`programs/pc1261/lc3.hex` は、COR 氏のコンパイラ LC-3(ポケコン・ジャーナル 1989 年 6 月号) です。1 バイトの整数だけを扱うゲーム向けのコンパイラで、BASIC のプログラムとして打ち込んだソースをマシン語に直して動かします。一覧の「LC-3 コンパイラ」を選ぶと、PRO モードで `NEW` と `EQU#43` を打って BASIC の置き場所を `5600` 番地からにし、コンパイラを `4000`〜`4901` 番地に書き込みます。そのあと PRO モードでソースを打ち込み、`CALL &4000` でコンパイル、`CALL &4900` で実行します。どちらも PRO モードで打ちます。記事の見本のうち 3 本 (`lc3-hello`、`lc3-move`、`lc3-chars`) と、このエミュレータのために書いたブロック崩し (`lc3-block`) は、選ぶとコンパイルから実行まで続けて行います。ブロック崩しは、左端のパドルを 8 (上) と 2 (下) で動かしてボールを打ち返し、右側の 40 個のブロックを消します。パドルの端で打つと斜めに、まんなかで打つとまっすぐに返ります。ボールは 3 つで、全部消すか、ボールがなくなると点数が出て、キーを押すともう一度始まります。コンパイラは、https://github.com/hiroog/LC3_Compiler にある `LC-3.bin` を元にしました。コンパイラと記事の見本のソースは COR 氏 (GitHub では hiroog 氏) の作品で、エミュレータの MIT ライセンスには含まれません。作者は 2018 年 1 月 3 日のブログの記事 (https://wlog.flatlib.jp/archive/1/2018-1-3) に「LC-3 の配布に関しては自由に行っていただいて構いません。ダンプリスト、バイナリ等、任意に掲載していただいて結構です。」と書いていて、それに従って同梱しています。

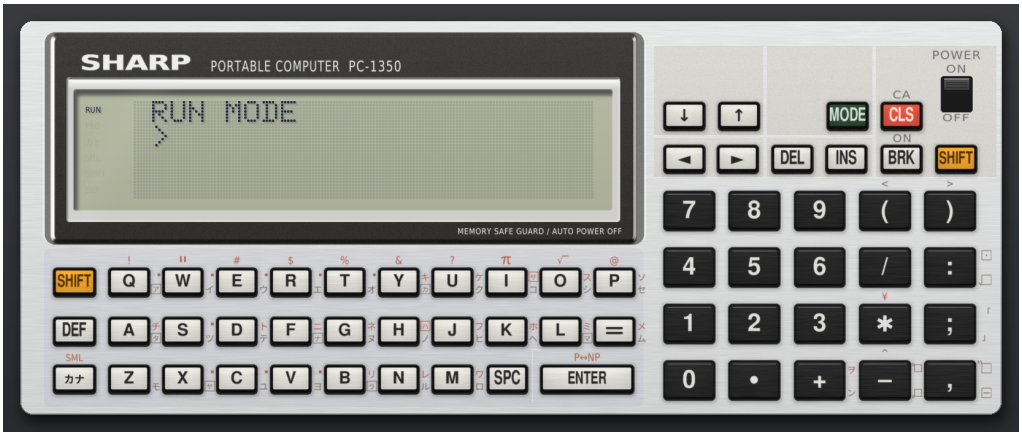
### 9.4　CPU の速さ

PC-1261 の CPU の水晶は 768kHz で、PC-1251(576kHz) の 4/3 倍の速さで動きます。ROM のテープを読む手続きは PC-1251 と同じ形で、1 ビットを読むときの繰り返しの回数などが PC-1251 の約 4/3 倍になっています。エミュレータを 576kHz として動かすと、ROM は自分で書いたテープを読めません。2ms と 512ms のタイマ、テープの 2kHz と 4kHz の音は、PC-1251 と同じ時間にしています。

## 10　PC-1350 として使う

PC-1350 は、150×32 ドットの液晶 (4 行 ×24 桁の字) を持つ機種です。CPU は同じ SC61860 ですが、ROM は PC-1251 とは別のもので、モードスイッチもありません。起動のときに`--model 1350` を付けると、PC-1350 として動きます。

```
$ uv run pc1251 --model 1350
$ uv run pc1251 --model 1350 --card 16
```




ROM は、実機から読み出したものを `cpu-1350.rom` と `bas-1350.rom` の名前で `rom` フォルダに置きます。カナの打てる国内向けの ROM で確かめました。`--overseas` をつけると輸出向けの配線で動き、カナの代わりに英小文字が入ります。筐体の絵は、PC-1350 を真上から撮った写真から描きました。

`.bas` や `.hex` のファイルを使う (5 節)、書き出す (5.9 節)、command / Ctrl + O などのショートカット (3.4 節)、DEF キーでの実行 (4.5 節)、カセットの音 (5.11 節) は、PC-1251 と同じように使えます。違うのは次のところです。

## 10.1　電源とモード

モードスイッチはありません。RUN モードと PRO モードは MODE キーで切り替えます。パソコンでは F1 で RUN モード、F2 で PRO モードにします (いまのモードと違えば MODE キーを押します)。いまのモードは、液晶の左端の `RUN・PRO` の表示記号で見ます。

本体の右上のスイッチは電源の ON・OFF です。マウスでクリックするかドラッグするか、F3 (ON)・F4 (OFF) で動かします。電源を切ってもプログラムは残ります。スイッチが OFF のあいだは、ON キー (Esc) では電源が入りません。

## 10.2　液晶

液晶は 150×32 ドットで、字は 4 行 ×24 桁です。液晶の左端に、`RUN`、`PRO`、カナ、`SML`、`SHIFT`、`DEF` の表示記号が縦に並びます。

## 10.3　キー

キーは PC-1350 の配置です。

- 括弧、/、:、;、, は数字の側のキーで、SHIFT を押さずに入ります。<、>、¥、^は、数字の側のキーを SHIFT を押してから打ちます。パソコンで記号を打つと、エミュレータがそのキー操作に直して打ちます。
- パソコンの Delete は DEL、Insert は INS、Home は CLS です。Backspace は ◀ と DEL で 1 字消します。

- カナキー (パソコンでは F8) でカナの入力に、SHIFT カナで英小文字の入力 (SML) に切り替わります。パソコンでカナ (全角のカタカナ、半角のカタカナ、ひらがな) を打つか貼り付けると、エミュレータがカナの入力に切り替えて打ち、打ち終わったら英字の入力に戻します。
- ファイルの BASIC の英小文字は、「"」で囲んだ文字列の中だけ SML に切り替えてそのまま打ち、ほかは大文字にします。命令は大文字でないと通りません。

PC-1350 の ROM は、キーを離してから次のキーを受け取れるようになるまでの時間が PC-1251 より長いので、貼り付けでは 1 打鍵に 0.25 秒ほどかけます。ファイルから読み込む BASIC のプログラムは、ほかの機種と同じく中間コードを RAM に直に置くので、すぐに入ります。

## 10.4　メモリと見本のプログラム

RAM は本体の 4KB で、プログラムに使えるのは 3070 バイト (MEM の値) です。起動のときに--card 8 か--card 16 をつけると、8KB か 16KB の RAM カード (CE-202M) を差した状態になり、使える大きさは 11262 バイトか 19454 バイトになります。終わるときの RAM は ~/.pc1251/ram-1350.bin(RAM カードを差したときは ram-1350-16k.bin のように大きさを付けた名前) に保存します。

見本のプログラムのうち、PC-1251 の番地を使うマシン語は一覧に出ません。PC-1350 用の見本は programs/pc1350 にあります。hello.bas は CURSOR で表示の位置を決めて、4 行の液晶にカナと英小文字を出します。ball.hex と ball.bas は、ボールが液晶のいちばん下で弾んでいちばん上まで上がりながら右へ進むマシン語で、地面に着くたびにつぶれて低い音が鳴ります (CALL &6100 で動かし、Esc (BRK) で BASIC に戻ります)。

# 付録 A　起動のオプション

| オプション | 意味 |
|---|---|
| プログラム (名前) | 読み込むプログラム (ファイルの場所か、置き場所にある名前) |
| `--run` | 読み込んだあと実行する |
| `--scale 数` | 拡大率 (既定は 1.0。画面に収まらなければ縮める) |
| `--fresh` | 保存した RAM を読まず、終わるときにも保存しない |
| `--mute` | 音を出さない |
| `--persist 数` | 液晶の残像の強さ (既定は 1.0、0 で残像なし) |
| `--type ファイル` | 起動後にファイルの中身をキーとして打ち込む |
| `--load ファイル` | 起動後に 16 進ダンプを RAM へ書き込む |
| `--export フォルダ` | 保存した RAM の BASIC のプログラムを`.bas` で書き出して終わる |
| `--model 機種` | `1251`(既定)、`1250`、`1255`、`1245`、`1260`、`1261`、`1262`、`1350` のどれか。<br>PC-1250・PC-1255 は 7 節、PC-1245 は 8 節、PC-1261 は 9 節、PC-1350 は 10 節 |
| `--card 大きさ` | PC-1350 に差す RAM カードの大きさ (KB)。`0`(既定)、`8`、`16` |
| `--overseas` | PC-1350 を輸出向けの配線で動かす (カナの代わりに英小文字が入る) |
| `--numpad` | 矢印キーを数字キーの 8・2・4・6 として押す状態で始める (command / Ctrl + K で切り替え) |
| `--tape ファイル` | カセットの音 (wav) をセットして始める (5.11 節) |
| `--state ファイル` | 保存した状態から始める (3.5 節) |
| `--norom` | ROM を使わず、マシン語の作業欄で動かす (6.1 節)。ROM が見つからなければ、つけなくても ROM なしで起動する |
| `--help` | オプションの一覧を出す |
| `--version` | 版を出して終わる |

| 環境変数 | 意味 |
|---|---|
| `PC1251_ROM` | ROM を置いたフォルダ |
| `PC1251_PROGRAMS` | 置き場所として先に探すフォルダ。いくつも書くときは、macOS では`:`、Windows では`;` で区切る |

# 付録 B　SHIFT で入る記号

パソコンで次の記号を打つと、右のキー操作に直して打ち込みます。

| 記号 | キー | 記号 | キー | 記号 | キー |
|---|---|---|---|---|---|
| ! | SHIFT+Q | & | SHIFT+Y | ( | SHIFT+ ↓ |
| " | SHIFT+W | ? | SHIFT+U | ) | SHIFT+ ↑ |
| # | SHIFT+E | : | SHIFT+I | @ | SHIFT+3 |
| $ | SHIFT+R | , | SHIFT+O | < | SHIFT+* |
| % | SHIFT+T | ; | SHIFT+P | > | SHIFT+- |
| ^ | SHIFT+/ | √ | SHIFT+. | π | SHIFT+0 |
| € | SHIFT++ | ¥ | SHIFT+6 | | |

PC-1245 では ( と ) が SHIFT+1 と SHIFT+2 になります (8 節)。表の€ は、指数を表す E(Exp) の代わりに使う字です。貼り付ける文字列やファイルの中では、€ の代わりに [E] と書いてもかまいません。英字の E(大文字でも小文字でも) は指数になりません。ポケコンにはバックスラッシュ (\) がないので、パソコンで\を打つか、ファイルや貼り付ける文字列に\があると、¥として打ちます (英語配列のキーボードや、¥のキーで\が入る設定でも¥が打てます)。どの機種でも同じです。

SHIFT + は指数の E(Exp)、SHIFT ◀ は 1 字消す (DEL)、SHIFT ▶ は 1 字あける (INS)、SHIFT ENTER は P ↔ NP(プリンタを使うかどうかの切り替え)、SHIFT CL は CA(すべて消す) です。

# 付録 C　メモリマップと入出力

| 番地 | 中身 |
|---|---|
| 0000-1FFF | CPU の内部 ROM(8KB) |
| 4000-7FFF | BASIC ROM(16KB) |
| B800-C7FF | RAM(4KB)。B000-B7FF は B800-BFFF と同じものが見える。PC-1245 は C000-C7FF(2KB) |
| F800-F87F | 液晶の RAM。F800-F8FF と同じものが F900-FFFF の 256 バイトごとに見える。PC-1245 は E800-EFFF にも見える |

- CPU は SC61860 で、命令サイクルは 192kHz(水晶 576kHz の 1/3) です。命令の意味は MAME の実装をもとにしましたが、CUP、CDN、TEST など、ROM や当時のプログラムの動きと合わない命令は変えています。資料にない命令 72h(LIIH) は 2 バイトの命令として扱います。54h(READM) は、56h(READ) と同じく、次の番地のバイトを (P) に読んでそのバイトを命令として残す 1 バイトの命令にしました。コンパイラ LC-3 がこの形で使っています。
- モードスイッチは、ポート IB のビット 3 を出して INB 命令で読みます。ビット 0 が RSV、ビット 1 が PRO、ビット 2 が OFF で、どれも立っていなければ RUN です。
- C ポートのビット 2 を立てると、CPU はキーが押されるか 512ms のタイマが変わるまで止まって待ちます。ROM はキーを待つあいだ、C ポートのビット 2 を立てて止まり、起きた回数を数えてオートパワーオフの時間を計ります。
- ブザーは C ポートのビット 4〜6 で鳴らします。BASIC の BEEP は内蔵の 4kHz の発振を使い、マシン語ではビットを直接切り替えて好きな高さの音を出せます。
- 液晶は、左半分の 60 列が F800〜F83B の順に、右半分が F87B から F840 へ逆の順に並びます。1 バイトが縦 7 ドットの 1 列で、ビット 0 が上です。表示記号は F83C〜F83E のビットです。

## PC-1250・PC-1255

RAM は PC-1250 が C000〜C7FF、PC-1255 が A000〜C7FF です。ほかは PC-1251 と同じです (7 節)。

### PC-1261

| 番地 | 中身 |
|---|---|
| 0000-1FFF | CPU の内部 ROM(8KB) |
| 2000-20FF、2800-28FF | 液晶の RAM。2100-27FF には 2000-20FF、2900-2FFF には 2800-28FF と同じものが見える |
| 4000-67FF | RAM(10KB)。PC-1260 は 5800-67FF(4KB) |
| 8000-FFFF | BASIC ROM(32KB) |

- 液晶の RAM は、どちらも 00〜3B が 1 行目、40〜7B が 2 行目で、2000 の側が左の 12 桁、2800 の側が右の 12 桁です。表示記号は 203D と 207C のビットです。2880〜28FF は表示に使わず、ROM が式の計算の途中の値を積むのに使います。
- 何もつながっていない番地を読むと、番地の上位バイトが返ります。
- CPU の水晶は 768kHz です。キーの並び、モードスイッチ、タイマ、ブザー、カセットのポートは PC-1251 と同じです。

### PC-1350

| 番地 | 中身 |
|---|---|
| 0000-1FFF | CPU の内部 ROM(8KB) |
| 2000-5FFF | RAM カード。8KB のカードは 4000-5FFF(2000-3FFF にも同じものが見える)、16KB のカードは 2000-5FFF |
| 6000-6FFF | 本体の RAM(4KB) |
| 7000-79FF | 液晶の RAM。ROM は液晶の RAM の使わないところと 7A00-7DFF を作業域に使う |
| 7E00-7FFF | キーのポート |
| 8000-FFFF | BASIC ROM(32KB) |

- 液晶の RAM は横 30 列ずつの 5 つの区画 (7000、7200、7400、7600、7800) に分かれ、各区画は上から 8 ドットずつの 4 段です。段の頭は、区画の頭から +00、+40、+1E、+5E です。1 バイトが縦 8 ドットで、ビット 0 が上です。表示記号は 783C のビットで、ビット 0 が SHIFT、1 が DEF、4 が RUN、5 が PRO、6 がカナ、7 が SML です。
- キーは 2 つの組に分かれます。1 つは 7E00〜7FFF に書いた値のビット 0〜6 を出して INA で読む組、もう 1 つは IA のビットを 1 本出して INA でほかのビットを読む組です。ON キーは TEST 08 で読みます。
- CPU の水晶は 768kHz です。

## 付録 D　実機と違うところ

- 実機と並べて動きを比べてはいません。命令の意味、キーの配置、ポートの使い方は、参考にしたエミュレータ (付録 F) と当時の資料により、わからないところは ROM のコードを読んで決めました。

- カセットテープの入出力は、実機用のテープを録音した wav を `CLOAD` で読み込めるところまで確かめました。エミュレータが書き出した wav を実機で読み込ませることは、まだ確かめていません (5.11 節)。
- プリンタの字の形は、本体の ROM の字の形で代わりにしています。印字の速さと紙送りのキーは再現していません (5.10 節)。
- ROM の版の違いは確かめていません。
- 内部 ROM が BASIC の `PEEK` でも読めます。資料では、実機の内部 ROM は `DATA(35h)` 命令でしか読めないとされています。

## 付録 E　ファイルの一覧

| 場所 | 中身 |
|---|---|
| `pc1251emu/` | エミュレータ本体 (CPU、メモリとポート、画面、音、プログラムの読み込み) |
| `programs/` | 見本のプログラム (機種ごとのものは `pc1251/`、`pc1245/`、`pc1261/`、`pc1350/`) |
| `rom/` | 自分で用意した ROM(2.4 節、PC-1245 は 8 節) |
| `tests/` | 動作の試験 (`uv run python tests/test_basic.py` など) |
| `doc/` | この手引きと『PC-インタープリタを読む』(`pcinterp.pdf`) |
| `~/.pc1251/` | 保存した RAM、画面の画像、プログラムの置き場所 |

## 付録 F　謝辞

このエミュレータは、次のエミュレータの実装を参考にして作りました。作者の方々にお礼申し上げます。

- MAME(`https://github.com/mamedev/mame`)
- PockEmul(`https://github.com/paulie-g/PockEmul`)
- PokecomGO(`https://github.com/digihori/pokecom`)
- Pokecom GO Plus(`https://github.com/digihori/pokecom-go-plus`)

このエミュレータの作成には Anthropic の Claude を利用しています。